

**Practical No: 1****Aim: Introduction to Excel**

1. Conditional Formatting: Perform conditional formatting on a dataset using various criteria.

**Steps:**

1. **Select the Data:** Highlight the range of cells you want to format.
2. **Go to Home Tab:** On the Excel Ribbon, click the **Home** tab.

| Student name | Marks obtain |
|--------------|--------------|
| Sudha        | 800          |
| Siya         | 900          |
| Jaya         | 850          |
| Sudhir       | 750          |
| Shyam        | 450          |
| Riya         | 500          |
| Jay          | 600          |
| Sai          | 950          |
| Mahesh       | 990          |
| Raj          | 750          |

3. **Click Conditional Formatting:** Look for the **Styles** group and click **Conditional Formatting**.

The screenshot shows the Excel interface with the 'Home' tab selected. In the 'Styles' group, the 'Conditional Formatting' button is highlighted. A task pane titled 'Conditional Formatting' is open, displaying a color scale from red (low) to green (high). The data table from the previous block is visible below, with the 'Marks obtain' column highlighted in green, indicating the range selected for formatting.

| Student name | Marks obtain |
|--------------|--------------|
| Sudha        | 800          |
| Siya         | 900          |
| Jaya         | 850          |
| Sudhir       | 750          |
| Shyam        | 450          |
| Riya         | 500          |
| Jay          | 600          |
| Sai          | 950          |
| Mahesh       | 990          |
| Raj          | 750          |

**4. Choose a Rule Type:**

- **Highlight Cell Rules:**

To highlight values Greater Than, Less Than, Between, or Equal to a specific number.

- **Top/Bottom Rules:** To highlight the Top 10 items, Top 10%, etc.

- **Data Bars/Color Scales:** To visually represent magnitude using gradients or bars.

**5. Configure the Rule:**

- Select the specific condition (e.g., "Greater Than").
- Enter the value.
- Choose the format style.
- **Apply:** Click **OK**.

| Student name | Marks obtain |
|--------------|--------------|
| Sudha        | ➡ 800        |
| Siya         | ⬆ 900        |
| Jaya         | ⬆ 850        |
| Sudhir       | ➡ 750        |
| Shyam        | ⬇ 450        |
| Riya         | ⬇ 500        |
| Jay          | ⬇ 600        |
| Sai          | ⬆ 950        |
| Mahesh       | ⬆ 990        |
| Raj          | ➡ 750        |

**Aim: Pivot Table**

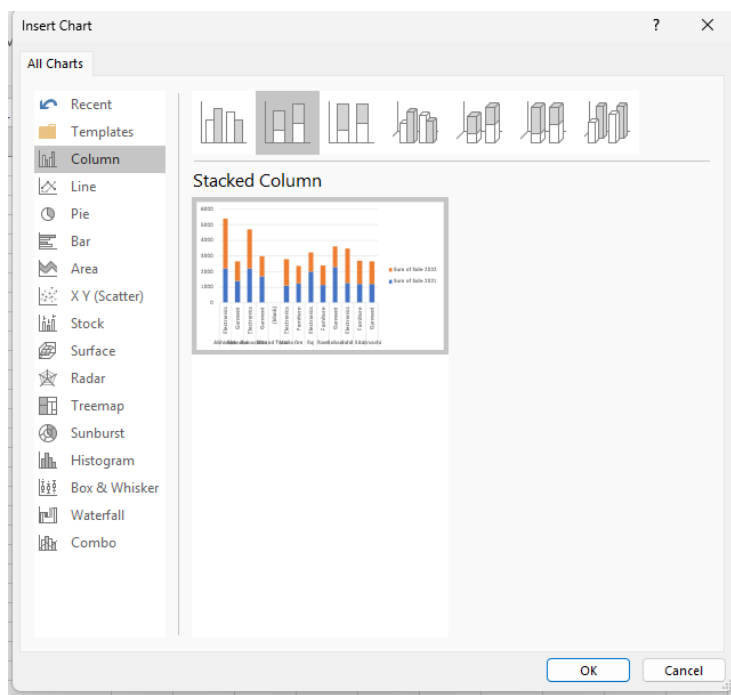
Create a pivot table to analyze and summarize data.

**Steps:**

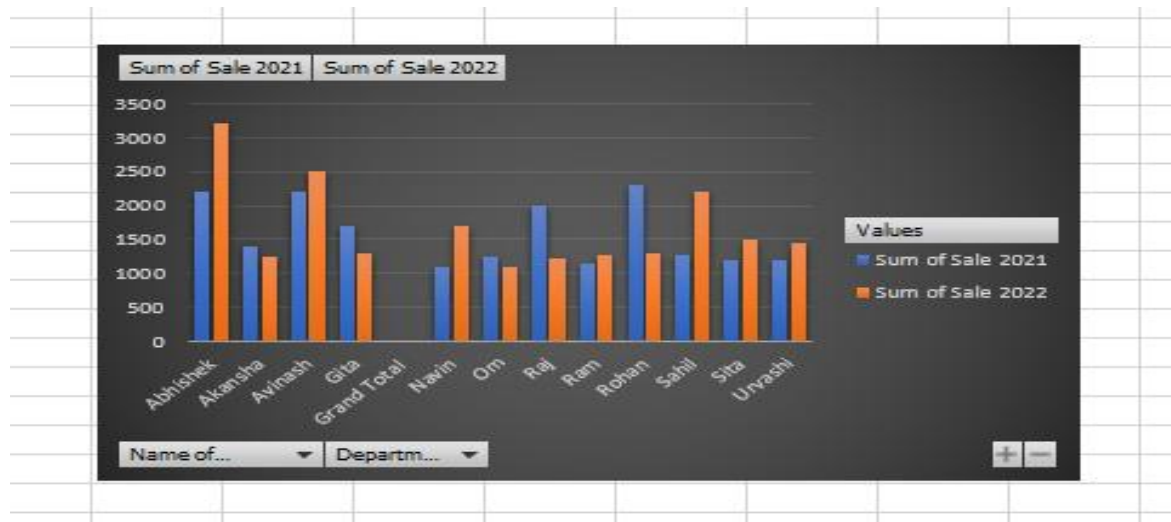
1. **Select Data:** Click anywhere inside your data range (ensure your data has headers like Name, Region, Sales).

| 3  | Row Labels  | Sum of Sale 2021 | Sum of Sale 2022 |
|----|-------------|------------------|------------------|
| 4  | Abhishek    | 2200             | 3200             |
| 5  | Electronic  | 2200             | 3200             |
| 6  | Akansha     | 1400             | 1240             |
| 7  | Garment     | 1400             | 1240             |
| 8  | Avinash     | 2200             | 2500             |
| 9  | Electronic  | 2200             | 2500             |
| 10 | Gita        | 1700             | 1300             |
| 11 | Garment     | 1700             | 1300             |
| 12 | Grand Total |                  |                  |
| 13 | (blank)     |                  |                  |
| 14 | Navin       | 1100             | 1700             |
| 15 | Electronic  | 1100             | 1700             |
| 16 | Om          | 1245             | 1100             |
| 17 | Furniture   | 1245             | 1100             |
| 18 | Raj         | 2000             | 1230             |
| 19 | Electronic  | 2000             | 1230             |
| 20 | Ram         | 1135             | 1270             |
| 21 | Furniture   | 1135             | 1270             |
| 22 | Rohan       | 2300             | 1300             |
| 23 | Garment     | 2300             | 1300             |
| 24 | Sahil       | 1270             | 2200             |
| 25 | Electronic  | 1270             | 2200             |
| 26 | Sita        | 1200             | 1500             |
| 27 | Furniture   | 1200             | 1500             |
| 28 | Urvashi     | 1200             | 1450             |
| 29 | Garment     | 1200             | 1450             |
| 30 | Grand Total | 18950            | 19990            |

2. **Insert Pivot Table:** Go to the **Insert** tab and click **PivotTable**.
3. **Create PivotTable Dialog:**
4. Ensure the **Table/Range** is correct.
5. Choose where to place it: **New Worksheet** (recommended).
6. Click **OK**.



7. **Configure Fields:** A "PivotTable Fields" pane will appear on the right.
8. **Rows area:** Drag a categorical field here (e.g., "Product Name" or "Region").
9. **Values area:** Drag a numerical field here (e.g., "Sales Amount"). It will automatically sum the values.
10. **Columns area:** (Optional) Drag another category here (e.g., "Year") to cross-tabulate.



**Aim:** VLOOKUP

Use VLOOKUP function to retrieve information from a different worksheet or table.

**Syntax:** =VLOOKUP (lookup\_value, table\_array, col\_index\_num, [range\_lookup])

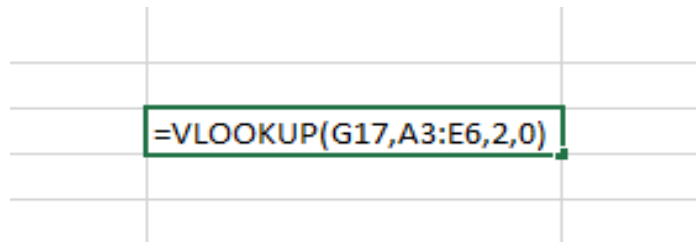
**Steps:**

1. **Select Cell:** Click part the cell where you want the result.

| SI. NO | EMPLOYEE NAME | DESIGNATION | BASIC  | NET SALARY |
|--------|---------------|-------------|--------|------------|
| 1      | Sudha         | Professor   | 100000 | 100000     |
| 2      | Siya          | Professor   | 11000  | 11000      |
| 3      | Jaya          | Professor   | 12000  | 12000      |
| 4      | Sudhir        | Professor   | 13000  | 13000      |
| 5      | Shyam         | Professor   | 14000  | 14000      |
| 6      | Riya          | Professor   | 15000  | 15000      |
| 7      | Jay           | Professor   | 16000  | 16000      |
| 8      | Sai           | Professor   | 17000  | 17000      |
| 9      | Mahesh        | Professor   | 18000  | 18000      |
| 10     | Raj           | Peon        | 10000  | 10000      |

current colum no, original serial no. box: expectef columns sallary box, column no, 0

2. **Start Formula:** Type = VLOOKUP (lookup\_value, table\_array, col\_index\_num, [range\_lookup])



3. **Argument 1: Lookup Value:** Click the cell that contains the unique ID you are searching for (e.g., A2, which might contain a Student ID).
4. **Argument 2: Table Array:**
  5. Navigate to the worksheet/table containing the source data.
  6. Select the entire range (e.g., **Sheet2! A1:D100**). **Important:** The first column of this selection **MUST** contain the IDs you are looking up.
7. **Argument 3: Col Index Num:** Enter the column number of the data you want to retrieve
8. Example: If you selected columns A to D, and want the name in column B, enter **2**.
9. **Argument 4: Range Lookup:** Type **FALSE** (or **0**) for an exact match. (Always recommended for IDs).
10. **Finish:** Close the parenthesis and press **Enter**.

| Sl. No. | Employee Name | Net Salary |  |
|---------|---------------|------------|--|
| 6       | Riya          | 15000      |  |
|         |               |            |  |
| Sl. No. | Employee Name | Net Salary |  |
| 10      | Raj           | 10000      |  |
|         |               |            |  |
| Sl. No. | Employee Name | Net Salary |  |
| 2       | Siya          | 11000      |  |
|         |               |            |  |
| Sl. No. | Employee Name | Net Salary |  |
| 3       | Sai           | 12000      |  |

**Aim: Goal Seek**

Perform what-if analysis using Goal Seek to determine input values for desired output.

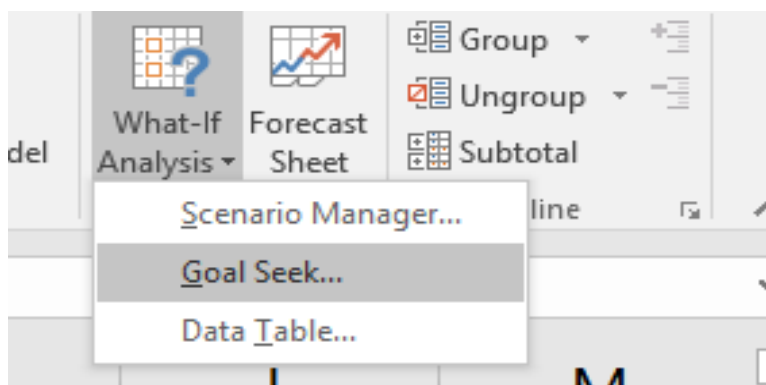
Goal Seek finds the necessary input value to achieve a specific result in a formula.

**Steps:**

1. **Prepare Data:** Ensure you have a cell with a formula (e.g., B5 contains  $=B1*B2*B3$  for simple interest, or a total sum).

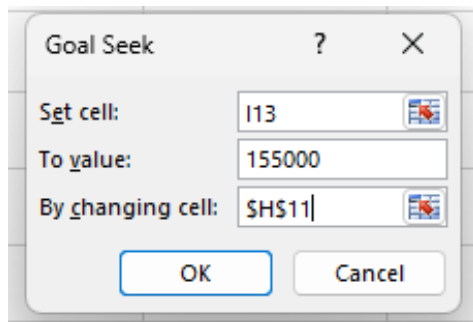
|              | Price   | Discount | Total         |
|--------------|---------|----------|---------------|
| <b>P1</b>    | 10000   | 20.00%   | 8000          |
| <b>P2</b>    | 20000   | 10.00%   | 18000         |
| <b>P3</b>    | 30000   | 15.00%   | 25500         |
| <b>P4</b>    | 40000   | 20.00%   | 32000         |
| <b>P5</b>    | 78235.3 | 15.00%   | 66500         |
| <b>Total</b> |         |          | <b>150000</b> |

2. **Open Goal Seek:**
3. Go to the **Data** tab.
4. Click **What-If Analysis** (in the Forecast group).
5. Select **Goal Seek**.



6. **Configure Parameters:**
7. **Set cell:** Reference the cell containing the \*formula\* (e.g., the Total Score or Final Price cell).
8. **To value:** Enter the \*result\* you want to achieve (e.g., 1000 or 75%).
9. **By changing cell:** Reference the cell containing the \*variable\* you want to adjust (e.g., "Exam Score" or "Discount Rate"). **Note:** This cell must contain a value, not a formula.

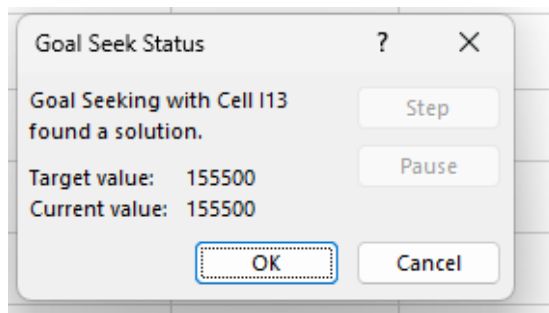
10. **Run:** Click **OK**.



The Goal Seek dialog box is shown with the following settings:

- Set cell:** I13
- To value:** 155000
- By changing cell:** \$H\$11

Buttons: OK, Cancel



The Goal Seek Status dialog box is shown with the following information:

- Goal Seeking with Cell I13 found a solution.**
- Target value:** 155500
- Current value:** 155500

Buttons: Step, Pause, OK, Cancel

11. **Result:** Excel will iterate and show the solution. Click **OK** to keep the change or **Cancel** to revert.

|              | Price   | Discount | Total         |
|--------------|---------|----------|---------------|
| <b>P1</b>    | 10000   | 20.00%   | 8000          |
| <b>P2</b>    | 20000   | 10.00%   | 18000         |
| <b>P3</b>    | 30000   | 15.00%   | 25500         |
| <b>P4</b>    | 40000   | 20.00%   | 32000         |
| <b>P5</b>    | 78235.3 | 7.97%    | 72000         |
| <b>Total</b> |         |          | <b>155500</b> |

**Practical No: 2**

**2A:** read data from CSV and JSON files into a data frame.

**CSV:**

Firstly you need one csv file.

**Code:**

```
getwd()
tuna <- read.csv("month.csv",TRUE,",")
class(tuna)
head (tuna)
```

**Output:**

```
> getwd()
[1] "C:/Users/dell/Documents"
> tuna <- read.csv("month.csv",TRUE,",")
> class(tuna)
[1] "data.frame"
> head(tuna)
  Name Abbreviation Numeric Numeric.2
1 January         Jan.         1         1
2 Feburary        Feb.         2         2
3 March           Mar.         3         3
4 April           Apr.         4         4
5 May             May          5         5
6 June            June         6         6
```

**JSON:****CODE:**

```
install.packages("rjson",dependencies = T)
library(rjson)
JsonData <- fromJSON(file = "sample.json")
print(JsonData)
JsonData_to_DataFrame <- as.data.frame(JsonData)
```



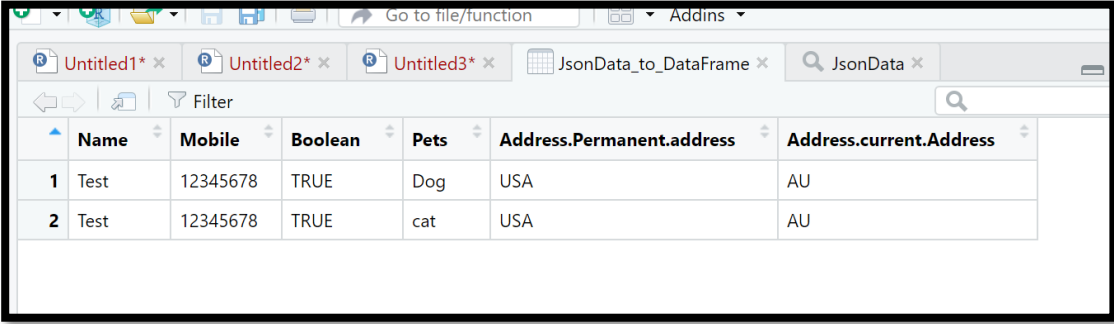
**OUTPUT:**

```
install.packages("rjson",dependencies = T)
WARNING: Rtools is required to build R packages but is
not currently installed. Please download and install the
appropriate version of Rtools before proceeding:
https://cran.rstudio.com/bin/windows/Rtools/
trying URL
'https://cran.rstudio.com/bin/windows/contrib/4.1/rjson_0
.2.21.zip'
Content type 'application/zip' length 772194 bytes (754
KB)
downloaded 754 KB
```

package 'rjson' successfully unpacked and MD5 sums checked

The downloaded binary packages are in  
C:\Users\de11\AppData\Local\Temp\RtmpKwf5zR\downloaded\_packages

```
> library(rjson)
> JsonData <- fromJSON(file = "sample.json")
> print(JsonData)
$Name
[1] "Test"
$Mobile
[1] 12345678
$Boolean
[1] TRUE
$Pets
[1] "Dog" "cat"
$Address
$Address$`Permanent address`
[1] "USA"
$Address$`current Address`
[1] "AU"
> JsonData_to_DataFrame <- as.data.frame(JsonData)
> View(JsonData_to_DataFrame)
> View(JsonData)
> View(JsonData_to_DataFrame)
```



|   | Name | Mobile   | Boolean | Pets | Address.Permanent.address | Address.current.Address |
|---|------|----------|---------|------|---------------------------|-------------------------|
| 1 | Test | 12345678 | TRUE    | Dog  | USA                       | AU                      |
| 2 | Test | 12345678 | TRUE    | cat  | USA                       | AU                      |

**2B:** Perform basic data pre-processing task such as handling missing values and outliers.

**Code: Handling Missing Values**

# Create a data frame with 5 rows and 3 columns

```
data <- data.frame(  
  A = c(1, 2, NA, 4, 5),  
  B = c(NA, 2, 3, NA, 5),  
  C = c(1, 2, 3, NA, NA)  
)
```

# View the resulting data frame

Data

**Output:**

```
  A B C  
1 1 NA 1  
2 2 2 2  
3 NA 3 3  
4 4 NA NA  
5 5 5 NA
```

# Finding missing values in data.

```
sum(is.na(data))
```

**Output:**

```
[1] 5
```

# Finding missing values column wise

```
colSums(is.na(data))
```

**Output:**

```
A B C  
1 2 2
```

**# Install and load the 'visdat' package**

```
install.packages("visdat")
```

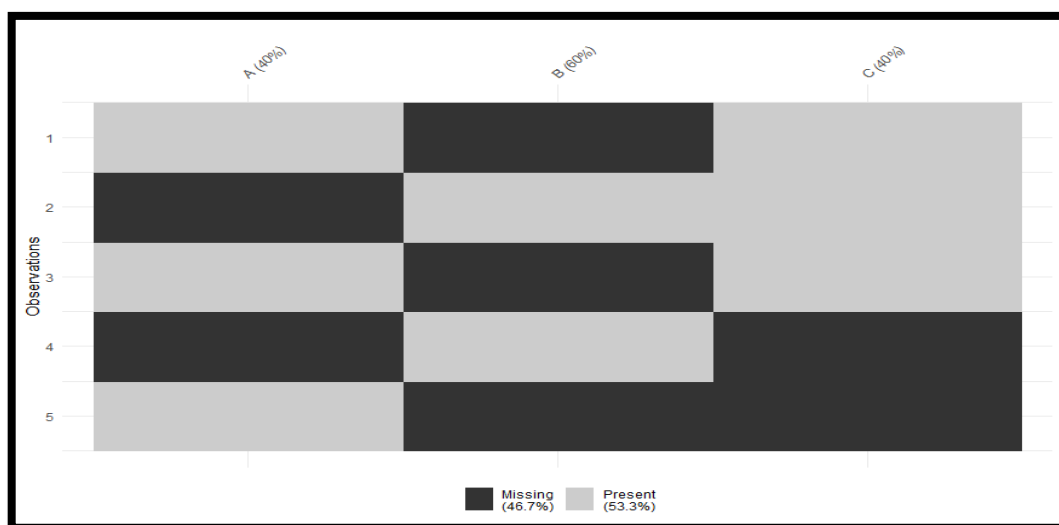
```
library(visdat)
```

**# Create a data frame with missing values**

```
data <- data.frame(  
  A = c(1, 2, NA, 4, 5),  
  B = c(NA, 2, 3, NA, 5),  
  C = c(1, 2, 3, NA, NA)  
)
```

**# Plot the missing value diagram**

```
vis_miss(data)
```

**Output:****# Remove missing values using na.omit function.**

```
data <- na.omit(data)
```

```
data
```

**Code: Handling Outliers:**

```
set.seed(937573)
```

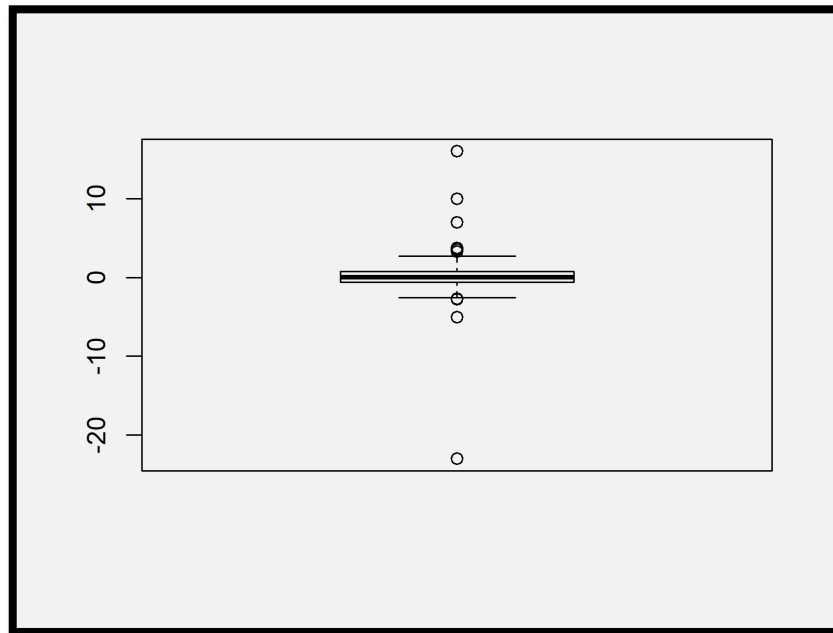
```
# Create randomly distributed data
```

```
x <- rnorm(1000)
```

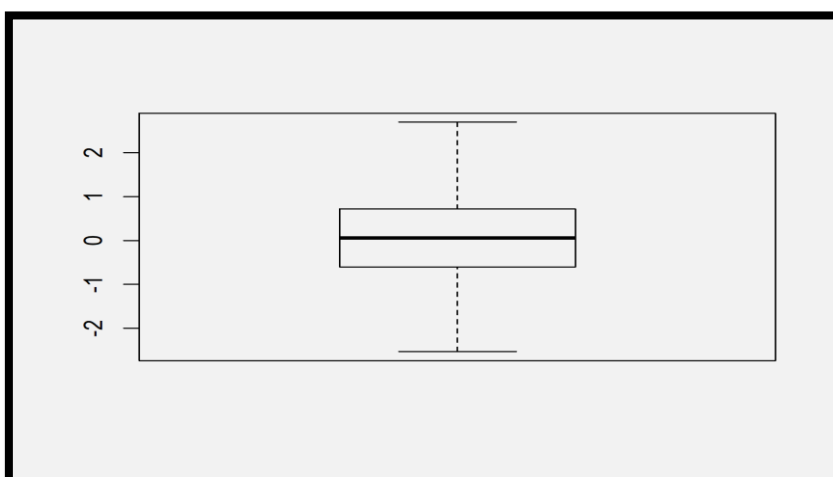
```
x[1:5] <- c(7, 10, -5, 16, -23)
```

```
# Insert outliers
```

```
x                                     # Print data
# [1]  7.000000000 10.000000000 -5.000000000 16.000000000 -23.000000000 -
0.413450746  0.801720348 ...
boxplot(x)                           # Create boxplot of all data
```



```
x_out_rm <- x[!x %in% boxplot.stats(x)$out] # Remove outliers
length(x) - length(x_out_rm)                # Count removed observations # 10
boxplot(x_out_rm)                           # Create boxplot without outliers
```



**2C:** Manipulate and transform data using functions like filtering, sorting, and grouping.

### **Filter Function:**

In the below code we used filter() function to fetch the data of players who scored more than 100 runs from the “stats” data frame.

#### **Code:**

```
# import dplyr package
```

```
library(dplyr)
```

```
# create a data frame
```

```
stats <- data.frame(player=c('A', 'B', 'C', 'D'),  
                    runs=c(100, 200, 408, 19),  
                    wickets=c(17, 20, NA, 5))
```

```
# fetch players who scored more
```

```
# than 100 runs
```

```
filter(stats, runs>100)
```

#### **Output:**

```
player runs wickets
```

```
1    B  200    20
```

```
2    C  408   NA
```

### **sort() function**

sort() function in R is used to sort a vector. By default, it sorts a vector in increasing order. To sort in descending order, add a “decreasing” parameter to the sort function.

#### **Code:**

```
# create a linear array
```

```
arr <- c(9, 8, 7, 6, 5, 4, 3, 2, 1)
```

**# use of sort function to sort array**

**# by default it is sorted in increasing order**

```
sort(arr)
```

**Output:**

```
> sort(arr)
```

```
[1] 1 2 3 4 5 6 7 8 9
```

### Grouping function:

**Group\_by() on a single column**

```
library(dplyr)
```

```
df = read.csv("Sample_Superstore.csv")
```

```
df_grp_region = df %>% group_by(Region) %>%
```

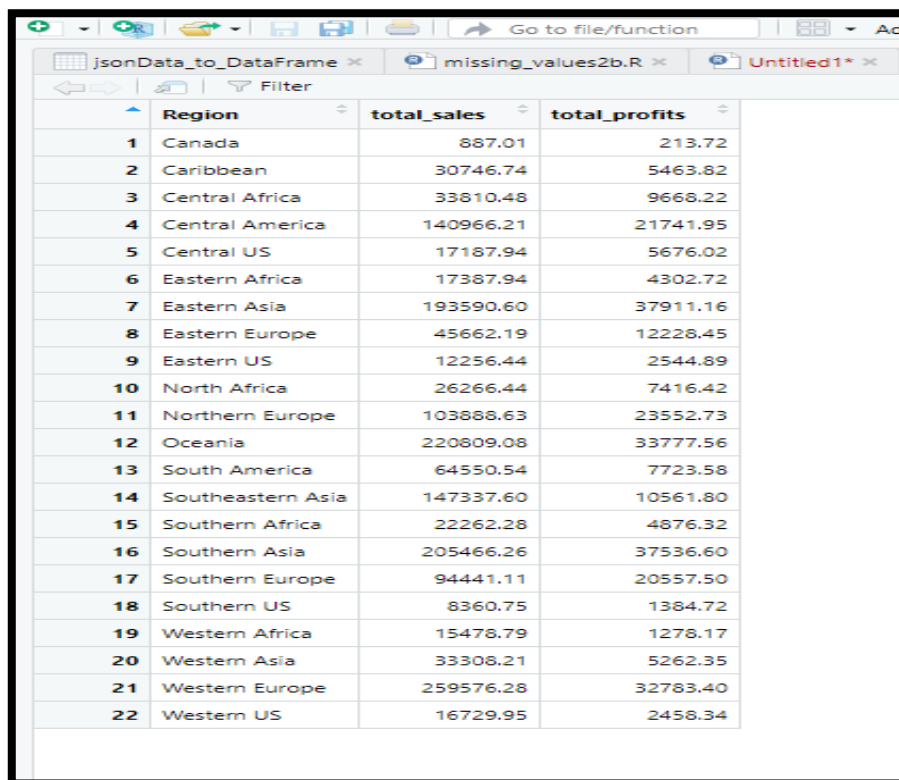
```
  summarise(total_sales = sum(Sales),
```

```
            total_profits = sum(Profit),
```

```
            .groups = 'drop')
```

```
View(df_grp_region)
```

**Output:**



|    | Region            | total_sales | total_profits |
|----|-------------------|-------------|---------------|
| 1  | Canada            | 887.01      | 213.72        |
| 2  | Caribbean         | 30746.74    | 5463.82       |
| 3  | Central Africa    | 33810.48    | 9668.22       |
| 4  | Central America   | 140966.21   | 21741.95      |
| 5  | Central US        | 17187.94    | 5676.02       |
| 6  | Eastern Africa    | 17387.94    | 4302.72       |
| 7  | Eastern Asia      | 193590.60   | 37911.16      |
| 8  | Eastern Europe    | 45662.19    | 12228.45      |
| 9  | Eastern US        | 12256.44    | 2544.89       |
| 10 | North Africa      | 26266.44    | 7416.42       |
| 11 | Northern Europe   | 103888.63   | 23552.73      |
| 12 | Oceania           | 220809.08   | 33777.56      |
| 13 | South America     | 64550.54    | 7723.58       |
| 14 | Southeastern Asia | 147337.60   | 10561.80      |
| 15 | Southern Africa   | 22262.28    | 4876.32       |
| 16 | Southern Asia     | 205466.26   | 37536.60      |
| 17 | Southern Europe   | 94441.11    | 20557.50      |
| 18 | Southern US       | 8360.75     | 1384.72       |
| 19 | Western Africa    | 15478.79    | 1278.17       |
| 20 | Western Asia      | 33308.21    | 5262.35       |
| 21 | Western Europe    | 259576.28   | 32783.40      |
| 22 | Western US        | 16729.95    | 2458.34       |

## Group\_by() on multiple columns

```
df = read.csv("Global_Superstore(CSV).csv")
df_grp_reg_cat = df %>% group_by(Region, Category) %>%
  summarise(total_Sales = sum(Sales),
            total_Profit = sum(Profit),
            .groups = 'drop')
View(df_grp_reg_cat)
```

|    | Region          | Category        | total_Sales | total_Profit |
|----|-----------------|-----------------|-------------|--------------|
| 1  | Canada          | Office Supplies | 887.01      | 213.72       |
| 2  | Caribbean       | Furniture       | 12935.22    | 3153.92      |
| 3  | Caribbean       | Office Supplies | 7971.20     | 1692.08      |
| 4  | Caribbean       | Technology      | 9840.32     | 617.82       |
| 5  | Central Africa  | Furniture       | 13578.48    | 3042.18      |
| 6  | Central Africa  | Office Supplies | 3129.48     | 586.98       |
| 7  | Central Africa  | Technology      | 17102.52    | 6039.06      |
| 8  | Central America | Furniture       | 43213.84    | 5904.96      |
| 9  | Central America | Office Supplies | 25707.02    | 3300.02      |
| 10 | Central America | Technology      | 72045.35    | 12536.97     |
| 11 | Central US      | Furniture       | 2085.15     | 161.74       |
| 12 | Central US      | Office Supplies | 12231.04    | 5609.39      |
| 13 | Central US      | Technology      | 2871.75     | -95.11       |
| 14 | Eastern Africa  | Furniture       | 3989.16     | 1461.84      |
| 15 | Eastern Africa  | Office Supplies | 4819.98     | 1100.28      |
| 16 | Eastern Africa  | Technology      | 8578.80     | 1740.60      |
| 17 | Eastern Asia    | Furniture       | 76786.81    | 9641.67      |
| 18 | Eastern Asia    | Office Supplies | 29599.25    | 6499.00      |
| 19 | Eastern Asia    | Technology      | 87204.54    | 21770.49     |
| 20 | Eastern Europe  | Furniture       | 20589.99    | 5733.33      |
| 21 | Eastern Europe  | Office Supplies | 8371.44     | 2571.90      |
| 22 | Eastern Europe  | Technology      | 16700.76    | 3923.22      |
| 23 | Eastern US      | Furniture       | 3258.53     | 244.52       |
| 24 | Eastern US      | Office Supplies | 7326.56     | 1740.33      |
| 25 | Eastern US      | Technology      | 1671.35     | 560.04       |
| 26 | North Africa    | Furniture       | 5496.12     | 1187.64      |
| 27 | North Africa    | Office Supplies | 5744.88     | 606.84       |
| 28 | North Africa    | Technology      | 15025.44    | 5621.94      |
| 29 | Northern Europe | Furniture       | 32728.42    | 6636.73      |
| 30 | Northern Europe | Office Supplies | 21317.61    | 5062.91      |
| 31 | Northern Europe | Technology      | 49842.60    | 11853.09     |
| 32 | Oceania         | Furniture       | 87320.35    | 6895.48      |
| 33 | Oceania         | Office Supplies | 36872.86    | 5579.05      |
| 34 | Oceania         | Technology      | 96615.87    | 21303.03     |
| 35 | South America   | Furniture       | 98765.43    | 2305.36      |

**Practical No. 03****Aim:** Feature Scaling and Dumification**A. Apply feature-scaling techniques like standardization and normalization to numerical features.****Creating a Dataset to apply feature scaling in R**  
**First, we need to create a data frame.****R**

```
# Age vector
age <- c(19,20,21,22,23,24,24,26,27)

# Salary vector
salary <- c(10000,20000,30000,40000,
            50000,60000,70000,80000,90000)

# Data frame created using age and salary
df <- data.frame( "Age" = age,
                  "Salary" = salary,
                  stringsAsFactors = FALSE)
```

df

**Output:**

```
Age Salary
1 19 10000
2 20 20000
3 21 30000
4 22 40000
5 23 50000
6 24 60000
7 24 70000
8 26 80000
9 27 90000
```

Once the dataset is created. Now we can start implementing Feature Scaling.

**By using General Formula****We know the formulas for both standardization and normalization. Let's apply them one by one.****implement standardization****R**

```
data <- data.frame(Age = rnorm(500, 50, 8),
                  Weight = rnorm(500, 80, 10))
data <- as.data.frame(sapply(df, function(x)
                             (x-mean(x))/sd(x)))
data
```



**Output:**

|   | Age          | Salary     |
|---|--------------|------------|
| 1 | -1.458333333 | -1.4605935 |
| 2 | -1.083333333 | -1.0954451 |
| 3 | -0.708333333 | -0.7302967 |
| 4 | -0.333333333 | -0.3651484 |
| 5 | 0.041666667  | 0.0000000  |
| 6 | 0.416666667  | 0.3651484  |
| 7 | 0.416666667  | 0.7302967  |
| 8 | 1.166666667  | 1.0954451  |
| 9 | 1.541666667  | 1.4605935  |

**implement normalization**

R

```
data2 <- data.frame(Age = rnorm(500, 50, 8),  
                    Weight = rnorm(500, 80, 10))  
data2 <- as.data.frame(sapply(df, function(x)  
                           (x-min(x))/(max(x)-min(x))))  
data2
```

**Output:**

|   | Age   | Salary |
|---|-------|--------|
| 1 | 0.000 | 0.000  |
| 2 | 0.125 | 0.125  |
| 3 | 0.250 | 0.250  |
| 4 | 0.375 | 0.375  |
| 5 | 0.500 | 0.500  |
| 6 | 0.625 | 0.625  |
| 7 | 0.625 | 0.750  |
| 8 | 0.875 | 0.875  |
| 9 | 1.000 | 1.000  |

## B. Perform feature dummification to convert categorical variables into numerical representations.

- **One-Hot Encoding**

The One-Hot A method of encoding category information into a binary matrix is called encoding. A column in the matrix is given to each distinct value in the categorical variable. The corresponding column will be given a value of 1, and all other columns will be given a value of 0, if the value is present in that particular row.

On consider the following data frame as an illustration:

```
gender <- c("male", "female", "male", "male", "female")
```

```
age <- c(23, 34, 52, 21, 19)
```

```
income <- c(50000, 70000, 80000, 45000, 55000)
```

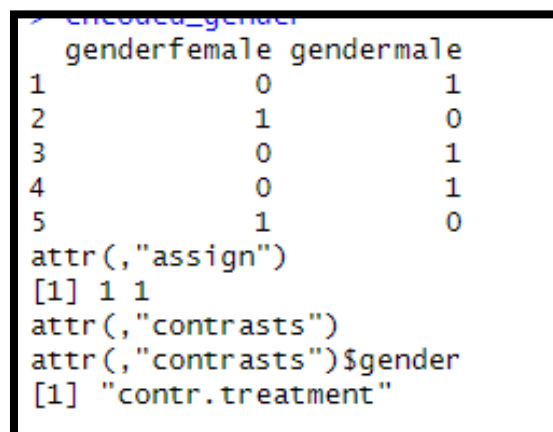
```
df <- data.frame(gender, age, income)
```

We can utilize the model to one-hot encode the gender column. function `model.matrix()`

```
encoded_gender <- model.matrix(~gender-1, data=df)
```

```
encoded_gender
```

**Output:**



```
> model.matrix(~gender-1, data=df)
  genderfemale gendermale
1          0           1
2          1           0
3          0           1
4          0           1
5          1           0
attr(,"assign")
[1] 1 1
attr(,"contrasts")
attr(,"contrasts")$gender
[1] "contr.treatment"
```

- **Label Encoding**

The Label encoding method is for encoding categorical variables that assigns the number value to each distinct value. For the instance, the numerical values 1, 2, and 3 might be assigned to a categorical variable with the three unique values of “red,” “green,” and “blue,” respectively.

The `factor()` function in R can be used to turn a category variable into a factor, that can subsequently be turned into integers using the `as.integer()` function.

Consider the following data frame as an illustration:

```
color <- c("red", "green", "blue", "blue", "red")
```

```
df <- data.frame(color)
```

We can use the following code to label encrypt the color column :

```
df$color <- as.integer(factor(df$color))  
df
```

**Output:**

```
[1] 3 2 1 1 3
```

- **Frequency Encoding**

The Frequency Each distinct value is assigned the frequency with which it occurs in the data when encoding categorical variables. The numerical values for each of these values may be 3, 4, and 2, respectively, if a categorical variable has three distinct values (red, green, and blue), and each of those values appears three, four, or two times.

On consider the following data frame as an illustration :

```
color <- c("red", "green", "blue", "blue", "red")  
df <- data.frame(color)
```

To frequency encode the color column, we can use the following code:

```
freq_count <- table(df$color)  
df$color <- match(df$color, names(freq_count))  
freq_count
```

**Output:**

```
> freq_count  
blue green red  
2 1 2
```

## Practical No 4

### Aim: Hypothesis Testing

#### Steps for conducting a Hypothesis Testing

Hypothesis testing is a statistical method used to make inferences about population parameters based on sample data. In R programming, you can perform various types of hypothesis tests, such as t-tests, chi-squared tests, and ANOVA, depending on the nature of your data and research question.

Here, I'll walk you through the steps for conducting a t-test (one of the most common hypothesis tests) in R. A t-test is used to compare the means of two groups, often in order to determine whether there's a significant difference between them.

#### 1. Prepare your data:

First, you'll need to have your data in R. You can either read data from a file (e.g., using `read.csv()`), or you can create vectors directly in R. For this example, I'll create two sample vectors for Group 1 and Group 2:

```
group1 <- c(12, 15, 17, 20, 22)
```

```
group2 <- c(18, 22, 25, 29, 30)
```

#### 2. State your null and alternative hypotheses:

In hypothesis testing, we start with a null hypothesis ( $H_0$ ) and an alternative hypothesis ( $H_1$ ). For a t-test, the null hypothesis is typically that there's no difference between the means of the two groups, while the alternative hypothesis is that there is a difference. In this example:

$H_0: \mu_1 = \mu_2$  (the means of Group 1 and Group 2 are equal)

$H_1: \mu_1 \neq \mu_2$  (the means of Group 1 and Group 2 are not equal)

#### 3. Perform the t-test:

Use the `t.test()` function to perform the t-test on your data. You can specify the type of t-test (independent samples, paired, or one-sample) with the appropriate arguments. In this case, we'll perform an independent samples t-test:

```
t_test_result <- t.test(group1, group2)
```

#### 4. Interpret the results:

The t-test result will include the t-value, degrees of freedom, and the p-value, among other information. The p-value is particularly important, as it helps you determine whether to accept or reject the null hypothesis. A common significance level (alpha) is 0.05. If the p-value is less than alpha, you can reject the null hypothesis, otherwise you fail to reject it.

```
print(t_test_result)
```

#### Output

```
> print(t_test_result)
```

Welch Two Sample t-test

data: group1 and group2

t = -2.6737, df = 7.6218, p-value = 0.02945

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-14.2119528 -0.9880472

sample estimates:

mean of x mean of y

17.2 24.8

#### 5. Make a decision:

Based on the p-value and your chosen significance level, make a decision about whether to reject or fail to reject the null hypothesis. If the p-value is less than 0.05, you would reject the null hypothesis and conclude that there is a significant difference between the means of the two groups.

Keep in mind that this example demonstrates the basic process of hypothesis testing using a t-test in R. Different tests and data may require additional steps, arguments, or functions. Be sure to consult R documentation and resources to ensure you're using the appropriate test and interpreting the results correctly.

**4B: Conduct a hypothesis test using appropriate statistical tests (e.g., t-test, chi square test).****t-test:****1. One-sample t-test:**

```
# Define data
data <- c(25, 30, 28, 35, 22, 29, 31)
# Set the known value to compare against
known_value <- 27
# Perform a one-sample t-test
result <- t.test(data, mu = known_value)
print(result)
```

**Output:**

One Sample t-test

```
data: data
t = 0.9905, df = 6, p-value = 0.3602
alternative hypothesis: true mean is not equal to 27
95 percent confidence interval:
 24.68938 32.45347
sample estimates:
mean of x
 28.57143
```

**2. Two-sample t-test:**

```
# Define two samples
group1 <- c(25, 30, 28, 35, 22, 29, 31)
group2 <- c(31, 34, 29, 35, 27, 32, 33)
# Perform a two-sample t-test
result <- t.test(group1, group2)
print(result)
```

**Output:**

Welch Two Sample t-test

data: group1 and group2

t = -1.5696, df = 10.5, p-value = 0.1461

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-7.231331 1.231331

sample estimates:

mean of x mean of y

28.57143 31.57143

**3. Paired t-test:**

```
# Define paired samples
```

```
pre_test <- c(25, 30, 28, 35, 22, 29, 31)
```

```
post_test <- c(31, 34, 29, 35, 27, 32, 33)
```

```
# Perform a paired t-test
```

```
result <- t.test(pre_test, post_test, paired = TRUE)
```

```
print(result)
```

**Output:**

Paired t-test

data: pre\_test and post\_test

t = -3.6742, df = 6, p-value = 0.0104

alternative hypothesis: true mean difference is not equal to 0

95 percent confidence interval:

-4.997895 -1.002105

sample estimates:

mean difference

### Chi-Square Test:

# Define a contingency table

```
data <- matrix(c(40, 60, 35, 55), nrow = 2, byrow = TRUE)
```

```
rownames(data) <- c("Male", "Female")
```

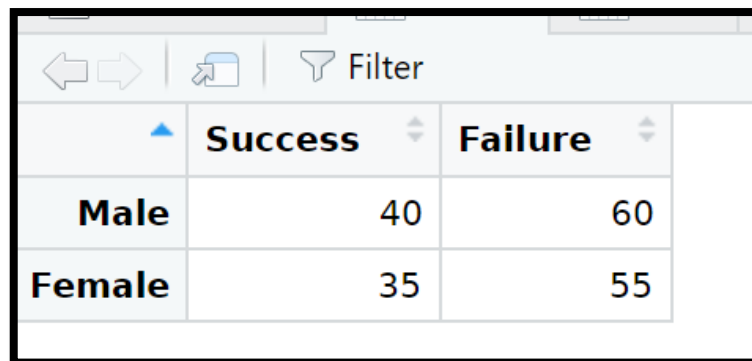
```
colnames(data) <- c("Success", "Failure")
```

# Perform a chi-squared test

```
result <- chisq.test(data)
```

```
print(result)
```

**Output:**



|        | Success | Failure |
|--------|---------|---------|
| Male   | 40      | 60      |
| Female | 35      | 55      |

Pearson's Chi-squared test with Yates' continuity correction

data: data

X-squared = 6.1192e-05, df = 1, p-value = 0.9938



## Practical No 5

**Aim:** ANOVA (Analysis of Variance)

Perform one-way ANOVA to compare means across multiple groups.

Conduct post-hoc tests to identify significant differences between group means.

**Code: PYTHON**

```
import scipy.stats as stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd
group1= [23,25,29,34,30]
group2= [43,19,22,20,18]
group3= [15,18,20,21,17]
group4= [28,24,30,29,26]
all_data=group1+group2+group3+group4
group_labels=['Group1']*len(group1) + ['Group2']*len(group2)
+['Group3']*len(group3) + ['Group4']*len(group4)
f_statistic, p_value=stats.f_oneway(group1,group2,group3,group4)
print("One -way ANOVA:")
print("F-staistic:", f_statistic)
print("P-value:", p_value)
tukey_results=pairwise_tukeyhsd(all_data,group_labels)
print("\nTukeys-Kramer post-hoc test:")
print(tukey_results)
```

**OUTPUT:**

```
One -way ANOVA:
F-staistic: 2.9319098457888493
P-value: 0.06535527272128736

Tukeys-Kramer post-hoc test:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====
group1 group2 meandiff p-adj  lower  upper reject
-----
Group1 Group2   -3.8 0.7439 -14.5241  6.9241  False
Group1 Group3  -10.0 0.072  -20.7241  0.7241  False
Group1 Group4   -0.8 0.9964 -11.5241  9.9241  False
Group2 Group3   -6.2 0.3784 -16.9241  4.5241  False
Group2 Group4    3.0 0.8532  -7.7241 13.7241  False
Group3 Group4    9.2 0.1064  -1.5241 19.9241  False
=====
```

**Practical no: 6****6A: Implement simple linear regression using a dataset.****What is Linear Regression?**

Linear Regression is a commonly used type of predictive analysis. Linear Regression is a statistical approach for modelling the relationship between a dependent variable and a given set of independent variables. It is predicted that a straight line can be used to approximate the relationship. The goal of linear regression is to identify the line that minimizes the discrepancies between the observed data points and the line's anticipated values.

**There are two types of linear regression.**

- Simple Linear Regression
- Multiple Linear Regression

**Simple Linear Regression:**

In Machine Learning Linear regression is one of the easiest and most popular Machine Learning algorithms.

- It is a statistical method that is used for predictive analysis.
- Linear regression makes predictions for continuous/real or numeric variables such as sales, salary, age, product price, etc.
- Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (x) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable changes according to the value of the independent variable.

## Implementation in R

In R programming, **lm()** function is used to create linear regression model.

**Syntax:** lm(formula)

### Parameter:

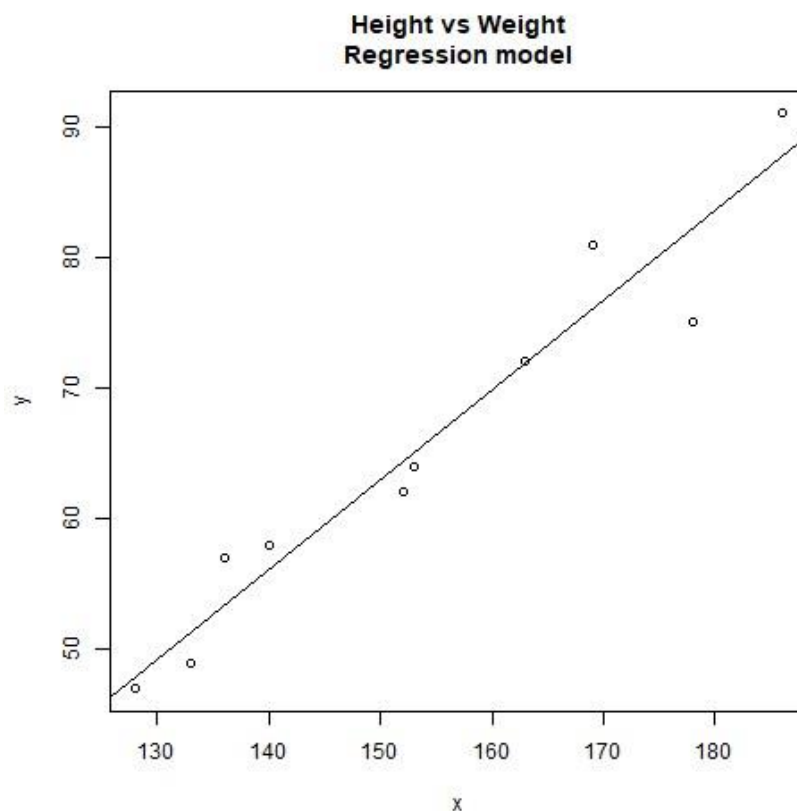
formula: represents the formula on which data has to be fitted To know about more optional parameters, use below command in console: help("lm")

Example: In this example, let us plot the linear regression line on the graph and predict the weight-based using height.

### Code:

```
# R program to illustrate
# Linear Regression
# Height vector
x <- c(153, 169, 140, 186, 128,
      136, 178, 163, 152, 133)
# Weight vector
y <- c(64, 81, 58, 91, 47, 57,
      75, 72, 62, 49)
# Create a linear regression model
model <- lm(y~x)
# Print regression model
print(model)
# Find the weight of a person
# With height 182
df <- data.frame(x = 182)
res <- predict(model, df)
cat("\nPredicted value of a person
      with height = 182")
print(res)
```

```
# Output to be present as PNG file
png(file = "linearReg.png")
# Plot
plot(x, y, main = "Height vs Weight Regression model")
abline(lm(y~x))
# Save the file.
dev.off()
```

**Output:**

**Call:** Using the “lm” function, we will be performing a regression analysis of “Salary” against “Years\_Exp” according to the formula displayed on this line.

**Residuals:** Each residual in the “Residuals” section denotes the difference between the actual salaries and predicted values. These values are unique to each observation in the data set. For instance, observation 1 has a residual of 463.1.

**Coefficients:** Linear regression coefficients are revealed within the contents of this section.

(Intercept): The estimated salary when Years\_Exp is zero is 30927, which represents the intercept for this case.

**Years\_Exp:** For every year of experience gained, the expected salary is estimated to increase by 7230 units according to the coefficient for “Years\_Exp”. This coefficient value suggests that each year of experience has a significant impact on the estimated salary.

**Estimate:** The model’s estimated coefficients can be found in this column.

**Std. Error:** “More precise estimates” can be deduced from smaller standard errors that are a gauge of the ambiguity that comes along with coefficient estimates.

**t value:** The coefficient estimate’s standard error distance from zero is measured by the t-value. Its purpose is to examine the likelihood of the coefficient being zero by testing the null hypothesis. A higher t-value’s absolute value indicates a higher possibility of statistical significance pertaining to the coefficient.

**Pr(>|t|):** This column provides the p-value associated with the t-value. The p-value indicates the probability of observing the t-statistic (or more extreme) under the null hypothesis that the coefficient is zero. In this case, the p-value for the intercept is 0.00144, and for “Years\_Exp,” it is 0.01482.

**Signif. codes:** These codes indicate the level of significance of the coefficients.

**Residual standard error:** This is a measure of the variability of the residuals. In this case, it’s 4944, which represents the typical difference between the actual salaries and the predicted salaries.

**Multiple R-squared:** R-squared ( $R^2$ ) is a measure of the goodness of fit of the model. It represents the proportion of the variance in the dependent variable that is explained by the independent variable(s). In this case, the R-squared is 0.7266, which means that approximately 72.66% of the variation in salaries can be explained by years of experience.

**Adjusted R-squared:** The adjusted R-squared adjusts the R-squared value based on the number of predictors in the model. It accounts for the complexity of the model. In this case, the adjusted R-squared is 0.6719.

**F-statistic:** The F-statistic is used to test the overall significance of the model. In this case, the F-statistic is 13.29 with 1 and 5 degrees of freedom, and the associated p-value is 0 In

summary, this linear regression analysis suggests that there is a significant relationship between years of experience (Years\_Exp) and salary (Salary). The model explains approximately 72.66% of the variance in salaries, and both the intercept and the coefficient for “Years\_Exp” are statistically significant at the 0.01 and 0.05 significance levels, respectively..01482. This p-value suggests that the model as a whole is statistically significant.

## **6B: Explore and interpret the regression model coefficients and goodness-of-fit measures.**

### **How to Extract Regression Coefficients from lm() Function in R**

```
#create data frame
```

```
df <- data.frame(rating=c(67, 75, 79, 85, 90, 96, 97),
```

```
    points=c(8, 12, 16, 15, 22, 28, 24),
```

```
    assists=c(4, 6, 6, 5, 3, 8, 7),
```

```
    rebounds=c(1, 4, 3, 3, 2, 6, 7))
```

```
#fit multiple linear regression model
```

```
model <- lm(rating ~ points + assists + rebounds, data=df)
```

```
#view only regression c
```

```
#Coefficients of model
```

```
model$coefficients
```

```
#view model summary
```

```
summary(model)
```

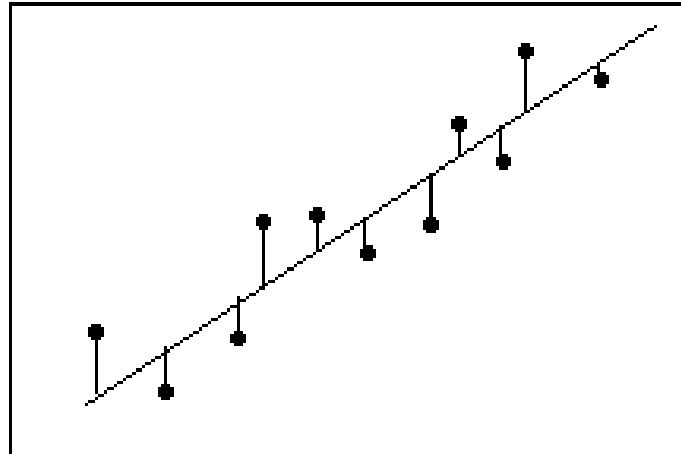
### **What Is Goodness-of-Fit for a Linear Model?**

Definition:  $\text{Residual} = \text{Observed value} - \text{Fitted value}$

Linear regression calculates an equation that minimizes the distance between the fitted line and all of the data points. Technically, ordinary least squares (OLS) regression minimizes the sum of the squared residuals.

In general, a model fits the data well if the differences between the observed values and the model's predicted values are small and unbiased.

Before you look at the statistical measures for goodness-of-fit, you should check the residual plots. Residual plots can reveal unwanted residual patterns that indicate biased results more effectively than numbers. When your residual plots pass muster, you can trust your numerical results and check the goodness-of-fit statistics.



```
# Load necessary libraries
```

```
library(ggplot2)
```

```
# Generate example data
```

```
set.seed(123) # for reproducibility
```

```
n <- 100
```

```
X <- rnorm(n)
```

```
Y <- 2*X + rnorm(n)
```

```
# Fit linear regression model
```

```
model <- lm(Y ~ X)
```

```
# Calculate predictions
```

```
predictions <- predict(model)
```

```
# Plot observed vs. predicted values
```

```
ggplot(data.frame(Observed = Y, Predicted = predictions), aes(x = Observed, y = Predicted))  
+ geom_point() + geom_abline(intercept = 0, slope = 1, color = "red") + # Add 45-degree  
line for reference
```

```
labs(x = "Observed", y = "Predicted", title = "Observed vs. Predicted")
```

### # Calculate goodness-of-fit measures

```
MSE <- mean((Y - predictions)^2)
```

```
RMSE <- sqrt(MSE)
```

```
MAE <- mean(abs(Y - predictions))
```

```
R_squared <- summary(model)$r.squared
```

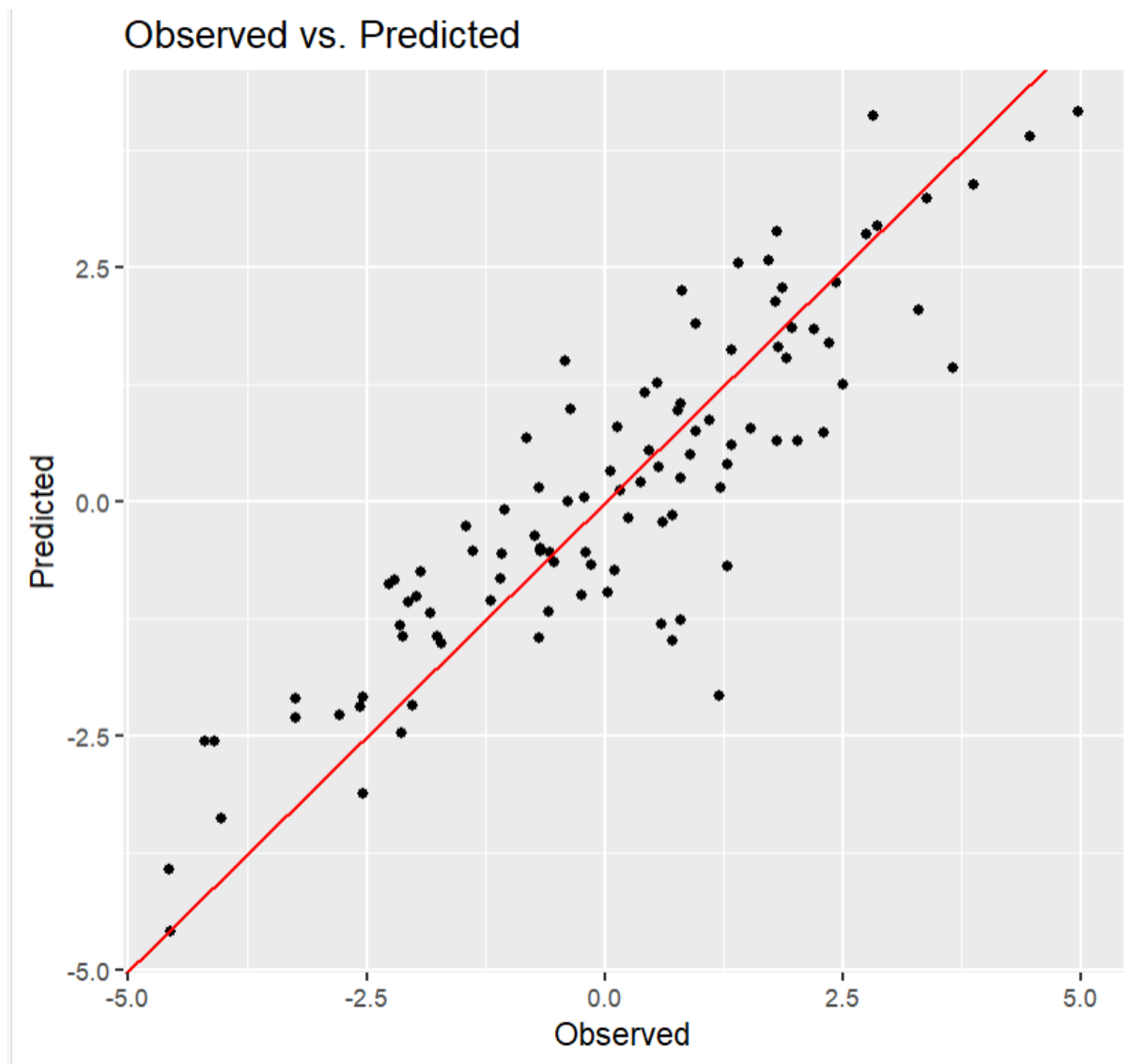
### # Print results

```
cat("Mean Squared Error (MSE):", MSE, "\n")
```

```
cat("Root Mean Squared Error (RMSE):", RMSE, "\n")
```

```
cat("Mean Absolute Error (MAE):", MAE, "\n")
```

```
cat("R-squared ( $R^2$ ):", R_squared, "\n")
```





## **6C: Extend the analysis to multiple linear regression and assess the impact of additional predictors.**

### **Multiple Linear Regression :**

It is the most common form of Linear Regression. Multiple Linear Regression basically describes how a single response variable Y depends linearly on a number of predictor variables.

The basic examples where Multiple Regression can be used are as follows:

1. The selling price of a house can depend on the desirability of the location, the number of bedrooms, the number of bathrooms, the year the house was built, the square footage of the lot, and a number of other factors.
2. The height of a child can depend on the height of the mother, the height of the father, nutrition, and environmental factors.

### **Implementation in R**

Multiple regression in R programming uses the same `lm()` function to create the model.

**Syntax:** `lm(formula, data)`

#### **Parameters:**

formula: represents the formula on which data has to be fitted

data: represents dataframe on which formula has to be applied

Example: Let us create a multiple regression model of air quality dataset present in R base package and plot the model on the graph.

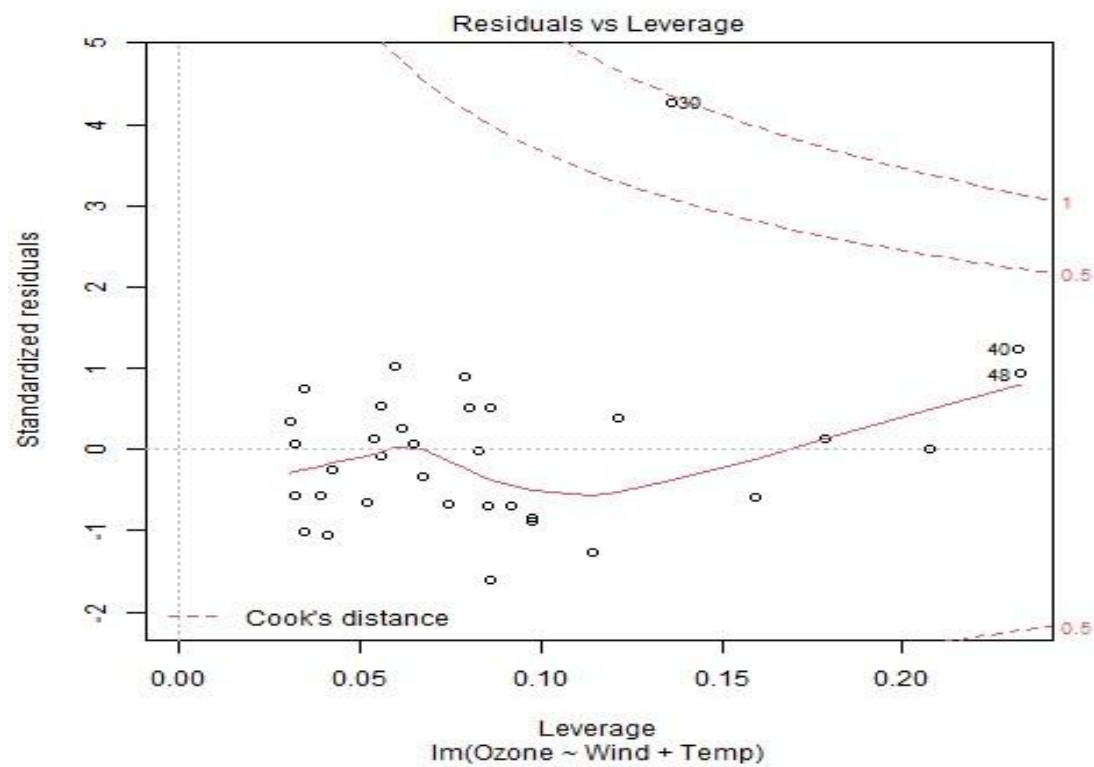
#### **Code:**

```
# R program to illustrate
# Multiple Linear Regression
# Using airquality dataset
input <- airquality[1:50,
                    c("Ozone", "Wind", "Temp")]

# Create regression model
model <- lm(Ozone~Wind + Temp,
            data = input)

# Print the regression model
cat("Regression model:\n")
print(model)
```

```
# Output to be present as PNG file  
png(file = "multipleRegGFG.png")  
# Plot  
plot(model)  
# Save the file.  
dev.off()
```



**Practical No : 07****Aim: Logistic Regression and Decision Tree.****A. Built a logistic regression model to predict a binary outcomes.**

# Installing the package

`install.packages("dplyr")`

# Loading package

`library(dplyr)`

# Summary of dataset in package

`summary(mtcars)`

# Installing the package

# For Logistic regression

`install.packages("caTools")`

# For ROC curve to evaluate model

`install.packages("ROCR")`

# Loading package

`library(caTools)``library(ROCR)`

# Splitting dataset

`split <- sample.split(mtcars, SplitRatio = 0.8)``split``train_reg <- subset(mtcars, split == "TRUE")``test_reg <- subset(mtcars, split == "FALSE")`

```
# Training model
logistic_model <- glm(vs ~ wt + disp,
                      data = train_reg,
                      family = "binomial")

logistic_model

# Summary
summary(logistic_model)

predict_reg <- predict(logistic_model,
                      test_reg, type = "response")

predict_reg

# Changing probabilities
predict_reg <- ifelse(predict_reg > 0.5, 1, 0)

# Evaluating model accuracy
# using confusion matrix
table(test_reg$vs, predict_reg)

missing_classerr <- mean(predict_reg != test_reg$vs)
print(paste('Accuracy =', 1 - missing_classerr))

# ROC-AUC Curve
ROCPred <- prediction(predict_reg, test_reg$vs)
ROCPer <- performance(ROCPred, measure = "tpr",
                     x.measure = "fpr")

auc <- performance(ROCPred, measure = "auc")
auc <- auc@y.values[[1]]

auc

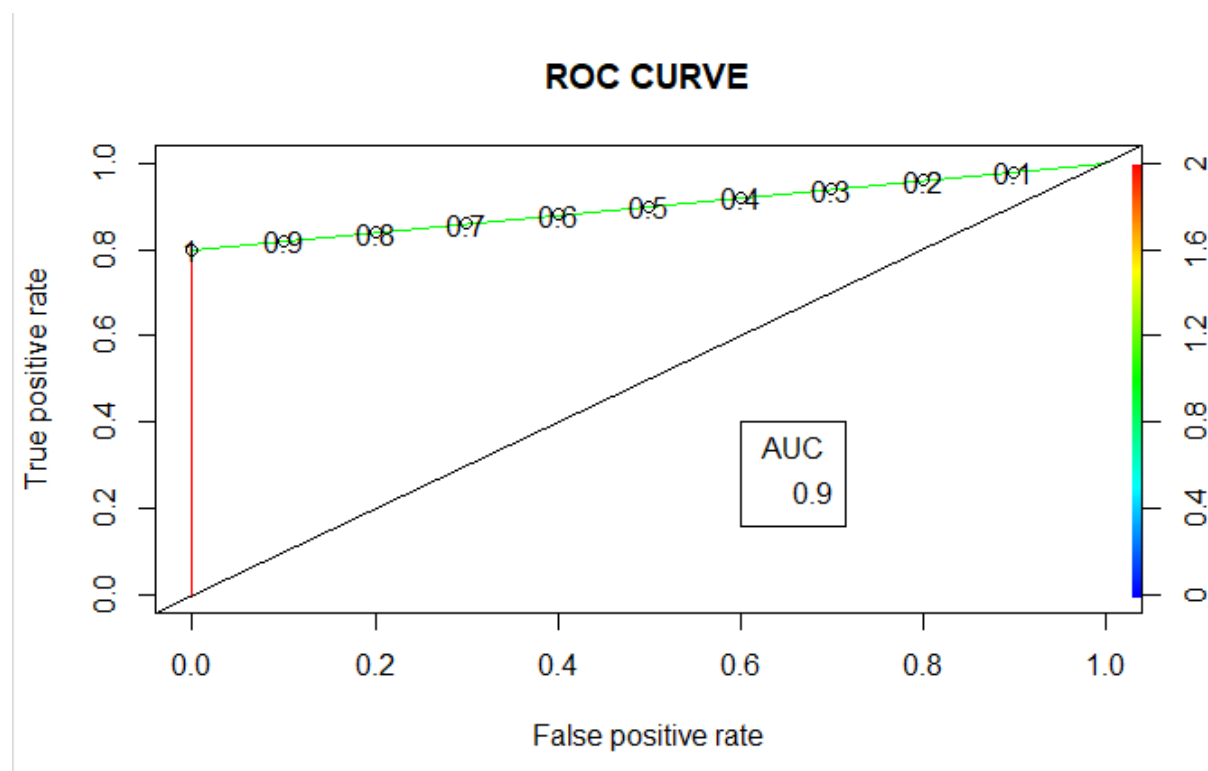
# Plotting curve
plot(ROCPer)

plot(ROCPer, colorize = TRUE,
     print.cutoffs.at = seq(0.1, by = 0.1),
     main = "ROC CURVE")

abline(a = 0, b = 1)

auc <- round(auc, 4)
```

```
legend(.6, .4, auc, title = "AUC", cex = 1)
```



**7B: Evaluate the model's performance using classification metrics (e.g., precision, recall).**

- **Calculating Recall by using the Mathematical formula**

The recall is nothing but the ratio between True Positives and the sum of the True Positive and False Negative.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Also, we can use another method to calculate recall, that is by using Metrics Package which has a 'recall' function to get the value. But first, we need to install the Metrics package in RStudio.

```
install.packages("Metrics")
```

```
install.packages("caret")
```

We type this command in the console and wait for a few minutes to let this package get installed. After installation, we can use this package.

```
# load metrics package  
library(Metrics)  
  
# Get the Recall  
recall = recall(actual = c(1,0,1,1,0),  
                 predicted = c(1,0,0,0,1))  
  
print(recall)
```

**Output:**

```
0.3333333
```

In these two ways, we can calculate Recall for our machine-learning model. Let us now see about the F1 score.

- **Precision and F1 – Score**

The F1-score is a measure of a model's performance that combines precision and recall. It is defined as the harmonic mean of precision and recall, where the best value is 1 and the worst value is 0.

There is another metric named Precision. In R, precision is a measure of a model's performance that tells you how many of the positive predictions made by the model are actually correct. It is calculated as the number of true positive predictions divided by the number of true positive and false positive predictions.

To compute precision and F1-score in R, we can use the caret package and the confusionMatrix() function. As we already have downloaded the caret package in RStudio we can directly implement this. Let us understand the precision and F1-score better with an example

```
# create predicted and actual class labels
predicted <- c(1, 1, 1, 0, 0)
actual <- c(1, 0, 1, 1, 1)
xtab <- table(predicted, actual)
library(caret)
cm <- caret::confusionMatrix(xtab)
cm
```

**Output:**

OUTPUT

Now we can calculate F1-Score and Precision using their formulas:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

R programming language provides us with another package called Metrics which has functions like precision(), f1\_score(), etc. We can directly calculate these evaluation metrics without having to use any formulas. Let us understand this by an example:

```
library(Metrics)
predicted <- c(1, 1, 0, 1, 0, 0, 0, 0, 1, 1)
actual <- c(1, 0, 0, 1, 0, 0, 1, 1, 1, 1)
# to calculate precision
Metrics::recall(predicted, actual)
# to calculate precision
Metrics::precision(predicted, actual)
# to calculate f1_score
Metrics::f1(predicted, actual)
```

**Output:**

0.8

0.6666667

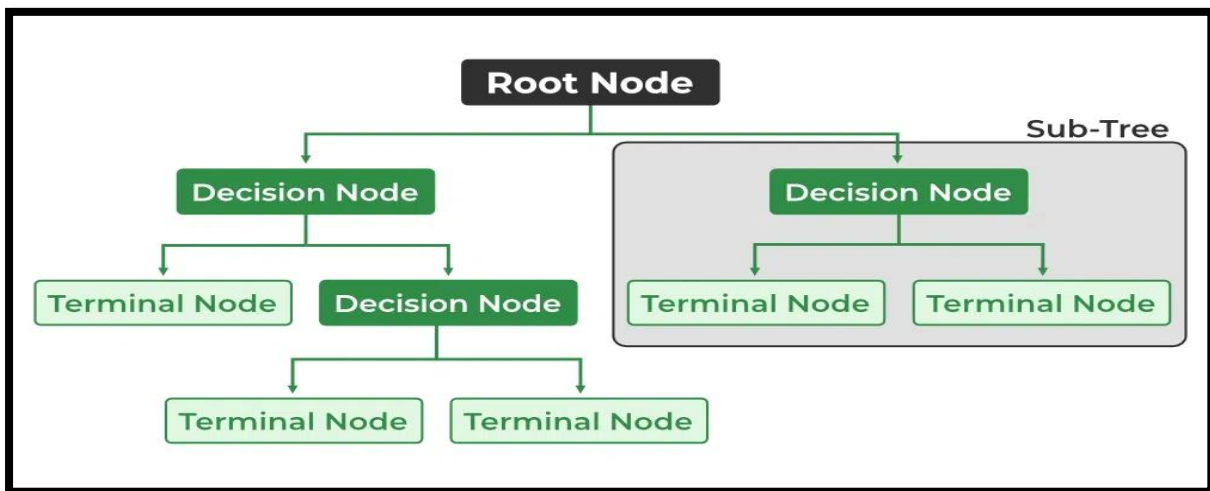
1

In this way, both these parameters are evaluated easily using the Metrics package.



**7C: Construct a decision tree model and interpret the decision rules for classification.**

A decision tree is a flowchart-like tree structure where each internal node denotes the feature, branches denote the rules and the leaf nodes denote the result of the algorithm. It is a versatile supervised machine-learning algorithm, which is used for both classification and regression problems. It is one of the very powerful algorithms. And it is also used in Random Forest to train on different subsets of training data, which makes random forest one of the most powerful algorithms in machine learning.

**Advantages of the Decision Tree:**

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

**Disadvantages of the Decision Tree:**

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the Random Forest algorithm.
- For more class labels, the computational complexity of the decision tree may increase.

**R – Decision Tree Example**

Let us now examine this concept with the help of an example, which in this case is the most widely used “readingSkills” dataset by visualizing a decision tree for it and examining its accuracy.

## Installing the required libraries

**Example 1:** In this example, let us predict the sepal width using the regression decision tree.

Step 1: Install the required package

```
# Install the package
```

```
install.packages("rpart")
```

Step 2: Load the package

```
# Load the package
```

```
library(rpart)
```

Step 3: Fit the model for decision tree for regression

```
# Create decision tree using regression
```

```
fit <- rpart(Sepal.Width ~ Sepal.Length +  
             Petal.Length + Petal.Width + Species,  
             method = "anova", data = iris)
```

Step 4: Plot the tree

```
# Output to be present as PNG file
```

```
png(file = "decTreeGFG.png", width = 600,height = 600)
```

```
# Plot
```

```
plot(fit, uniform = TRUE,  
      main = "Sepal Width Decision  
            Tree using Regression")
```

```
text(fit, use.n = TRUE, cex = .7)
```

```
# Saving the file
```

```
dev.off()
```

Step 5: Print the decision tree model

```
# Print model
```

```
print(fit)
```

Step 6: Predicting the sepal width

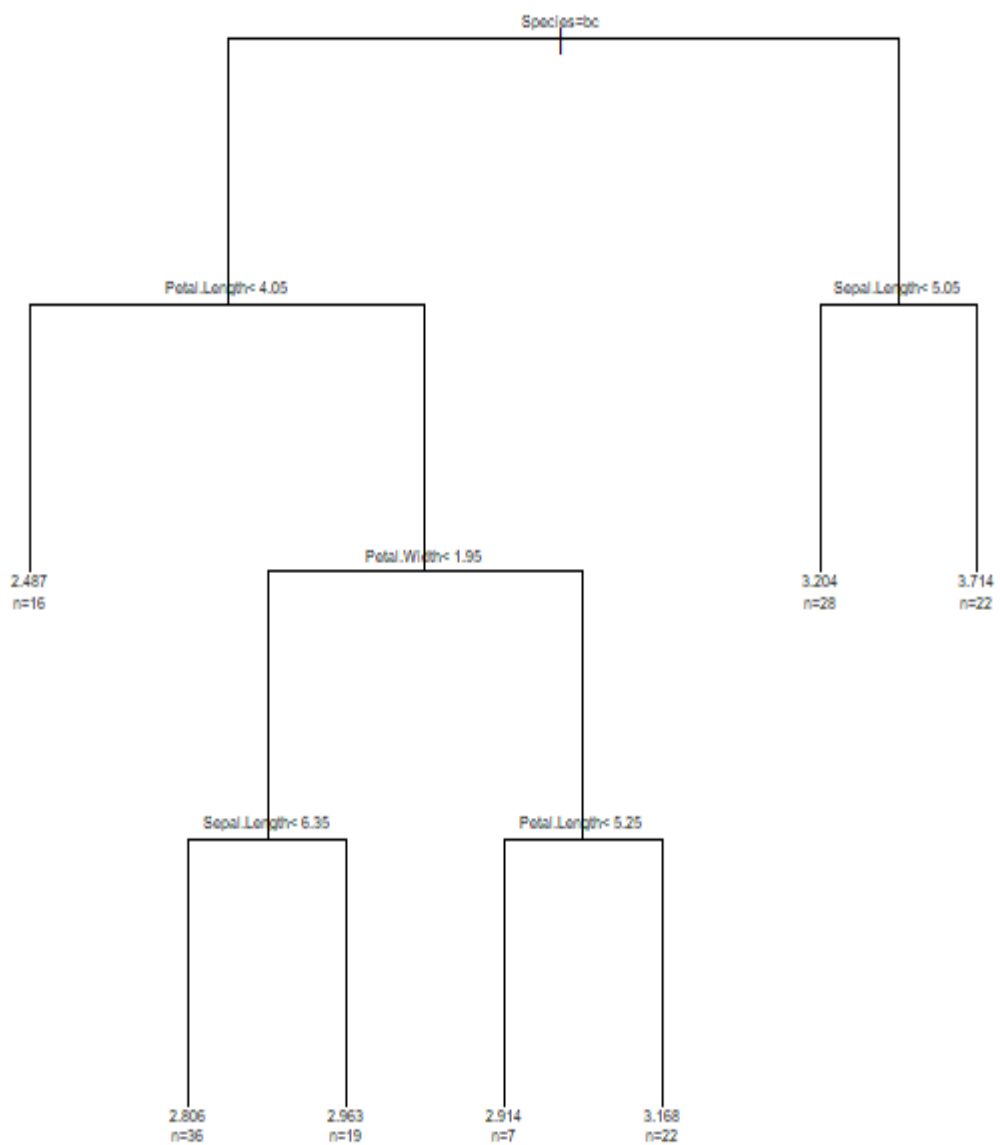
```
# Create test data
```

```
df <- data.frame (Species = 'versicolor',  
                  Sepal.Length = 5.1, Petal.Length = 4.5, Petal.Width = 1.4)
```

```
# Predicting sepal width  
# using testing data and model  
# method anova is used for regression  
cat("Predicted value:\n")  
predict(fit, df, method = "anova")
```

**Output:**

### Sepal Width Decision Tree using Regression



output graph

n= 150

node), split, n, deviance, yval

\* denotes terminal node

- 1) root 150 28.3069300 3.057333
- 2) Species=versicolor, virginica 100 10.9616000 2.872000
- 4) Petal.Length=4.05 84 7.3480950 2.945238
- 10) Petal.Width< 1.95 55 3.4920000 2.860000
- 20) Sepal.Length=6.35 19 0.6242105 2.963158 \*
- 11) Petal.Width>=1.95 29 2.6986210 3.106897
- 22) Petal.Length=5.25 22 2.0277270 3.168182 \*
- 3) Species=setosa 50 7.0408000 3.428000
- 6) Sepal.Length=5.05 22 1.7859090 3.713636 \*

Predicted value:

1

2.805556

**Practical No :8**  
**K-Means Clustering**

**8A: Apply the K-Means algorithm to group similar data points into clusters.**

**Syntax: kmeans(x, centers, nstart)**

where,

x represents numeric matrix or data frame object

centers represents the K value or distinct cluster centers

nstart represents number of random sets to be chosen

```
# Library required for fviz_cluster function
```

```
install.packages("factoextra")
```

```
library(factoextra)
```

```
# Loading dataset
```

```
df <- mtcars
```

```
# Omitting any NA values
```

```
df <- na.omit(df)
```

```
# Scaling dataset
```

```
df <- scale(df)
```

```
# output to be present as PNG file
```

```
png(file = "KMeansExample11.png")
```

```
km <- kmeans(df, centers = 4, nstart = 25)
```

```
# Visualize the clusters
```

```
fviz_cluster(km, data = df)
```

```
# saving the file
```

```
dev.off()
```

```
# output to be present as PNG file
```

```
png(file = "KMeansExample12.png")
```

```
km <- kmeans(df, centers = 5, nstart = 25)
```

```
# Visualize the clusters
```

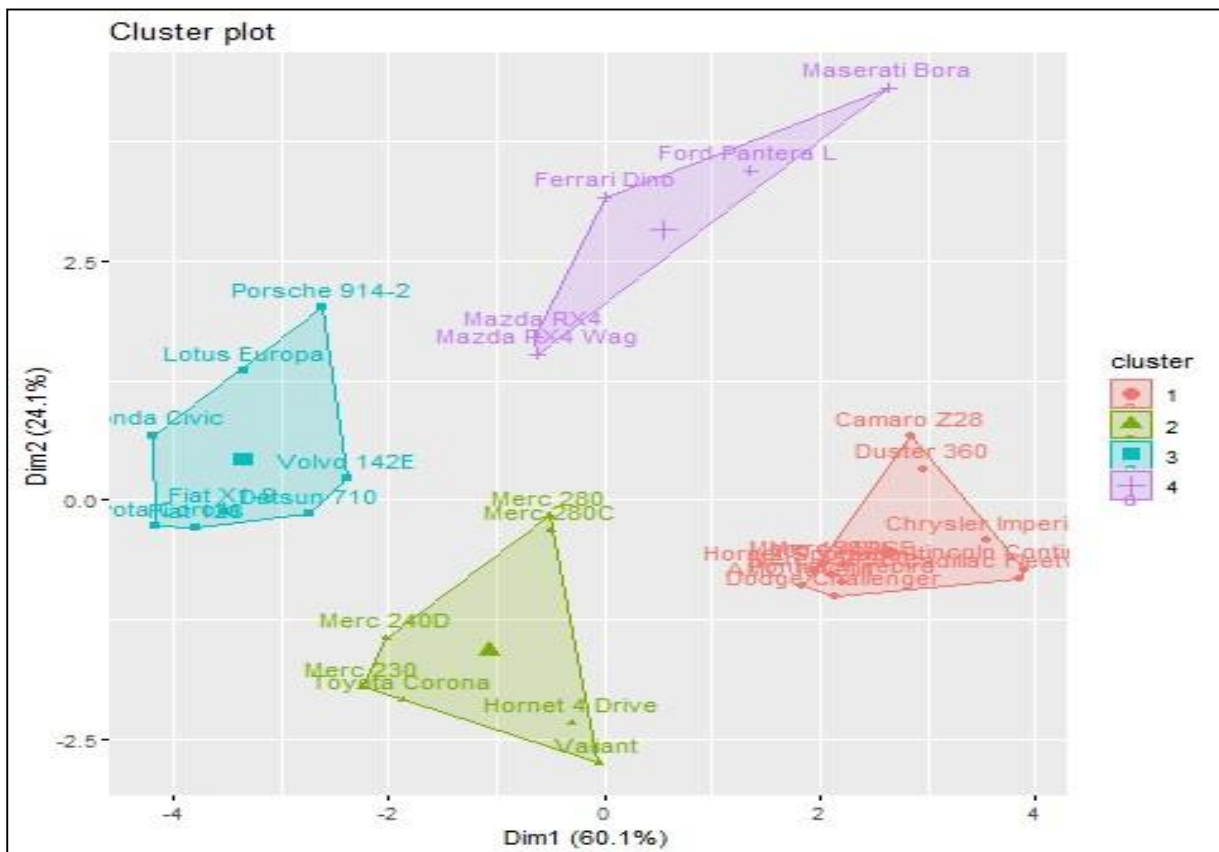
```
fviz_cluster(km, data = df)
```

```
# saving the file
```

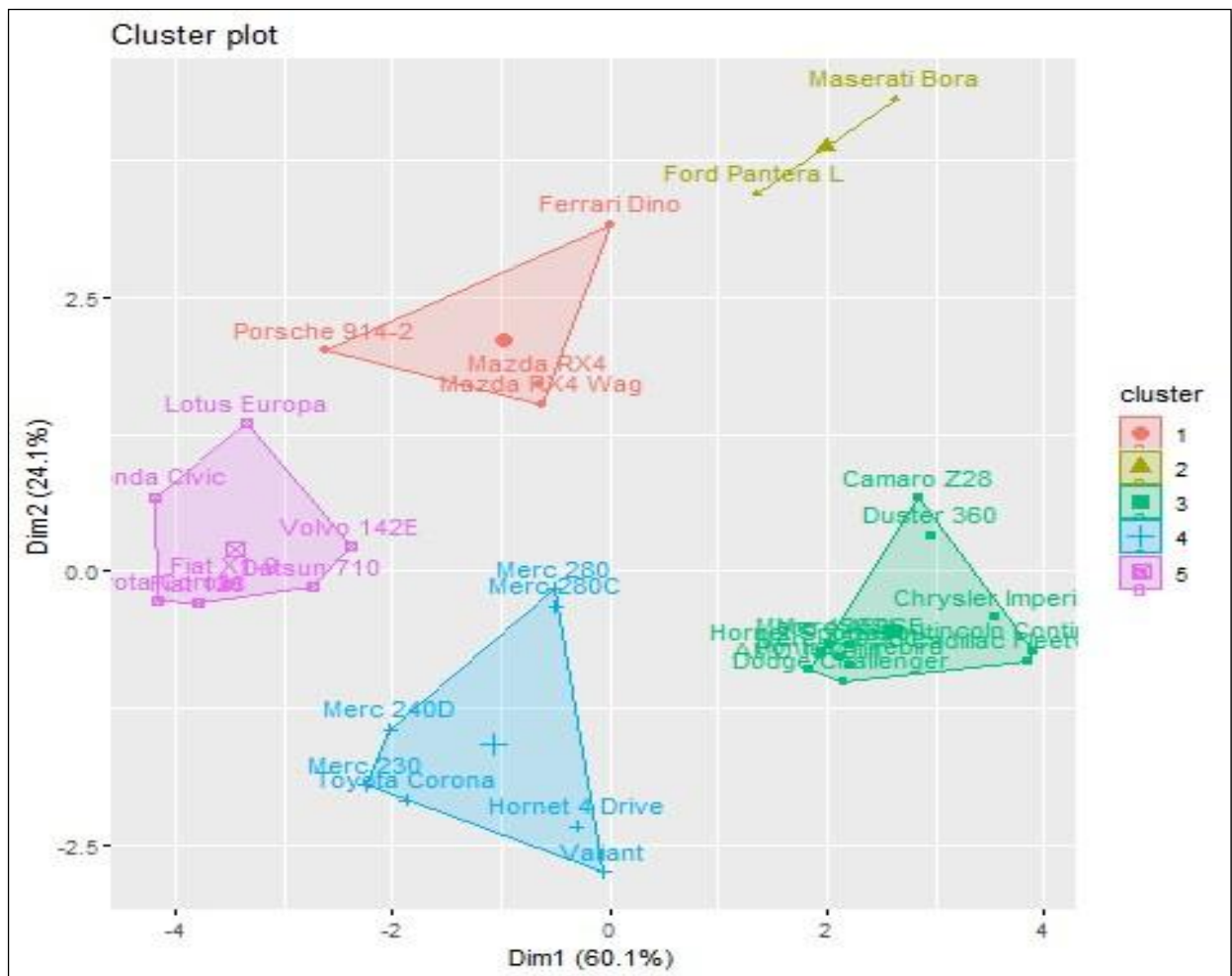
```
dev.off()
```

**Output:**

**When k = 4**



## When k = 5



Using the `fviz_cluster` function from the `factoextra` package, this code applies k-means clustering to the `mtcars` dataset using two different values of centers (4 and 5) and then saves the cluster visualizations as PNG files.

The clustering technique uses the information from the `mtcars` dataset, which includes details on various automobile models including the number of cylinders, horsepower, and miles per gallon. The `scale` function is used to scale the data to have a zero mean and unit variance, and the `na.omit` function is used to delete any rows with missing values.

Then, to improve the chance of discovering the global optimum, the `means` function is used to execute k-means clustering with 4 and 5 clusters. The `fviz_cluster` function, which plots the data points coloured by cluster membership and also displays the cluster centers, is then used to see the resulting cluster assignments.

The `png` and `dev.off` functions are then used to save the generated plots as PNG files.

To save the files to the correct location on your computer, you might need to modify the file paths or file names used in the `png` function.

**8B: Determine the optimal number of clusters using elbow method analysis.**

One of the most common ways to choose a value for K is known as the elbow method, which involves creating a plot with the number of clusters on the x-axis and the total within sum of squares on the y-axis and then identifying where an “elbow” or bend appears in the plot.

The point on the x-axis where the “elbow” occurs tells us the optimal number of clusters to use in the k-means clustering algorithm.

**Example: Using the Elbow Method in R**

For this example we'll use the USArrests dataset built into R, which contains the number of arrests per 100,000 residents in each U.S. state in 1973 for Murder, Assault, and Rape along with the percentage of the population in each state living in urban areas, UrbanPop.

The following code shows how to load the dataset, remove rows with missing values, and scale each variable in the dataset to have a mean of 0 and standard deviation of 1:

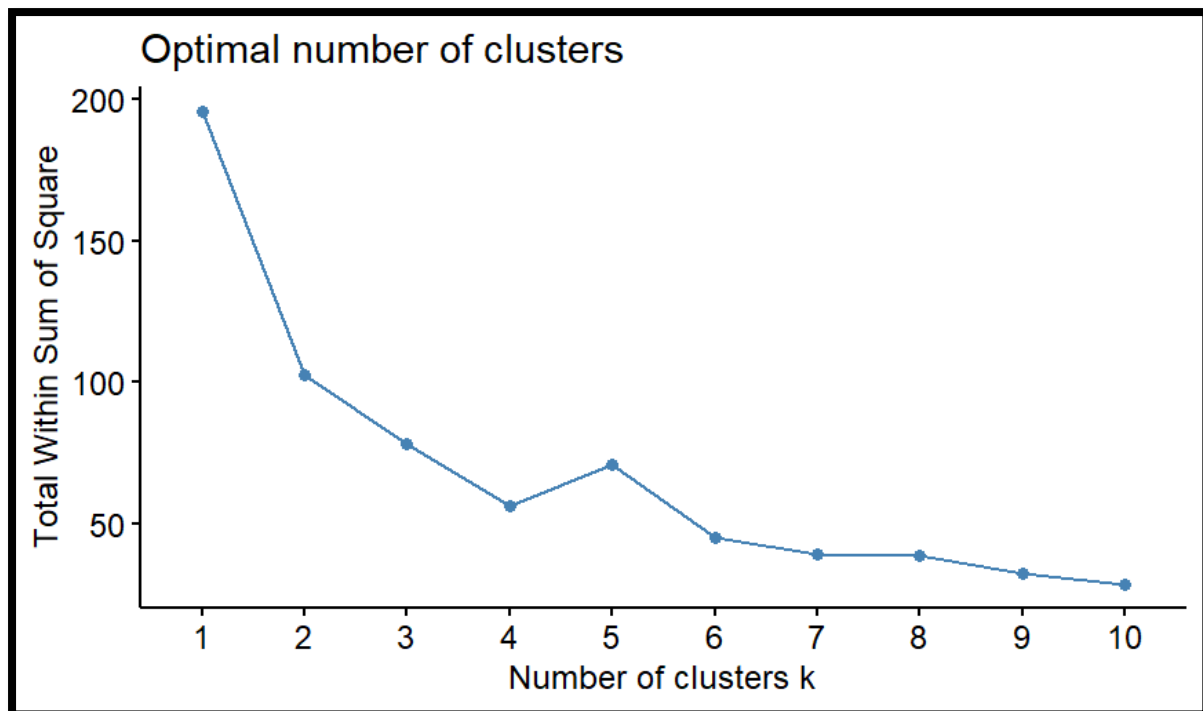
```
#load data
df <- USArrests
#remove rows with missing values
df <- na.omit(df)
#scale each variable to have a mean of 0 and sd of 1
df <- scale(df)
#view first six rows of dataset
head(df)
```

To find the optimal number of clusters to use in the k-means algorithm, we'll use the `fviz_nbclust()` function from the `factoextra` package to create a plot of the number of clusters vs. the total within sum of squares:

```
library(cluster)
library(factoextra)

#create plot of number of clusters vs total within sum of squares
fviz_nbclust(df, kmeans,
method = "wss")
```





In this plot it appears that there is an “elbow” or bend at  $k = 4$  clusters. This is the point where the total within sum of squares begins to level off.

This tells us that the optimal number of clusters to use in the k-means algorithm is 4.

We can proceed to use the `kmeans()` function from the cluster package to perform k-means clustering on the dataset using the optimal value for  $k$  of 4:

```
#make this example reproducible  
set.seed(1)
```

```
#perform k-means clustering with k = 4 clusters
```

```
km <- kmeans(df, centers = 4, nstart = 25)
```

```
#view results
```

```
km
```

**From the results we can see that:**

16 states were assigned to the first cluster

13 states were assigned to the second cluster

13 states were assigned to the third cluster

8 states were assigned to the fourth cluster

**We can also append the cluster assignments of each state back to the original dataset:**

```
#add cluster assignment to original data
```

```
final_data <- cbind(USArrests, cluster = km$cluster)
```

```
#view final data
```

```
head(final_data)
```

## Practical No: 9

### 9A: Evaluate the explained variance and select the appropriate number of principal components.

#### The Dataset

The dataset mtcars(motor trend car road test) comprises fuel consumption and 10 aspects of

automobile design and performance for 32 automobiles. It comes pre-installed with dplyr package in R.

```
# Installing required package
```

```
install.packages("dplyr")
```

```
# Loading the package
```

```
library(dplyr)
```

```
# Importing excel file
```

```
install.packages("mtcars")
```

```
str(mtcars)
```

```
# Loading Data
```

```
data(mtcars)
```

```
# Apply PCA using prcomp function
```

```
# Need to scale / Normalize as
```

```
# PCA depends on distance measure
```

```
my_pca = prcomp(mtcars, scale = TRUE,
```

```
                center = TRUE, retx = T)
```

```
names(my_pca)
```

```
# Summary
```

```
summary(my_pca)
```

```
my_pca
```

```
# View the principal component loading
```

```
# my_pca$rotation[1:5, 1:4]
```

```
my_pca$rotation
# See the principal components
dim(my_pca$x)
my_pca$x
# Plotting the resultant principal components
# The parameter scale = 0 ensures that arrows
# are scaled to represent the loadings
biplot(my_pca, main = "Biplot", scale = 0)
# Compute standard deviation
my_pca$sdev
# Compute variance
my_pca.var= my_pca$sdev ^ 2
my_pca.var
# Proportion of variance for a scree plot
propve = my_pca.var / sum(my_pca.var)
propve
# Plot variance explained for each principal component
plot(propve, xlab = "principal component",
      ylab = "Proportion of Variance Explained",
      ylim = c(0, 1), type = "b",
      main = "Scree Plot")
# Plot the cumulative proportion of variance explained
plot(cumsum(propve),
      xlab = "Principal Component",
      ylab = "Cumulative Proportion of Variance Explained",s
      ylim = c(0, 1), type = "b")
# Find Top n principal component
```

```

# which will atleast cover 90 % variance of dimension
which(cumsum(propve) >= 0.9)[1]

# Predict mpg using first 4 new Principal Components

# Add a training set with principal components
train.data = data.frame(displacement = mtcars$displacement, my_pca$x[, 1:4])

# Running a Decision tree algorithm

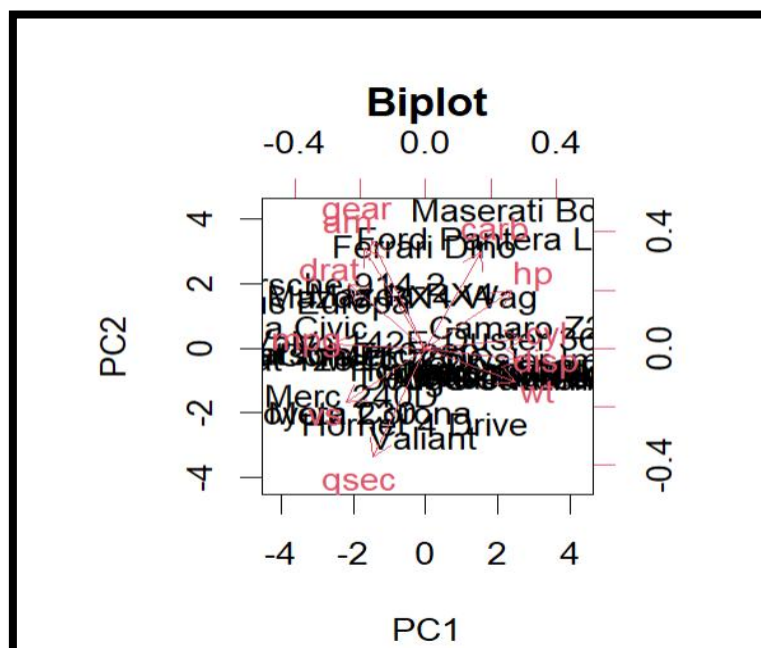
## Installing and loading packages
install.packages("rpart")
install.packages("rpart.plot")
library(rpart)
library(rpart.plot)

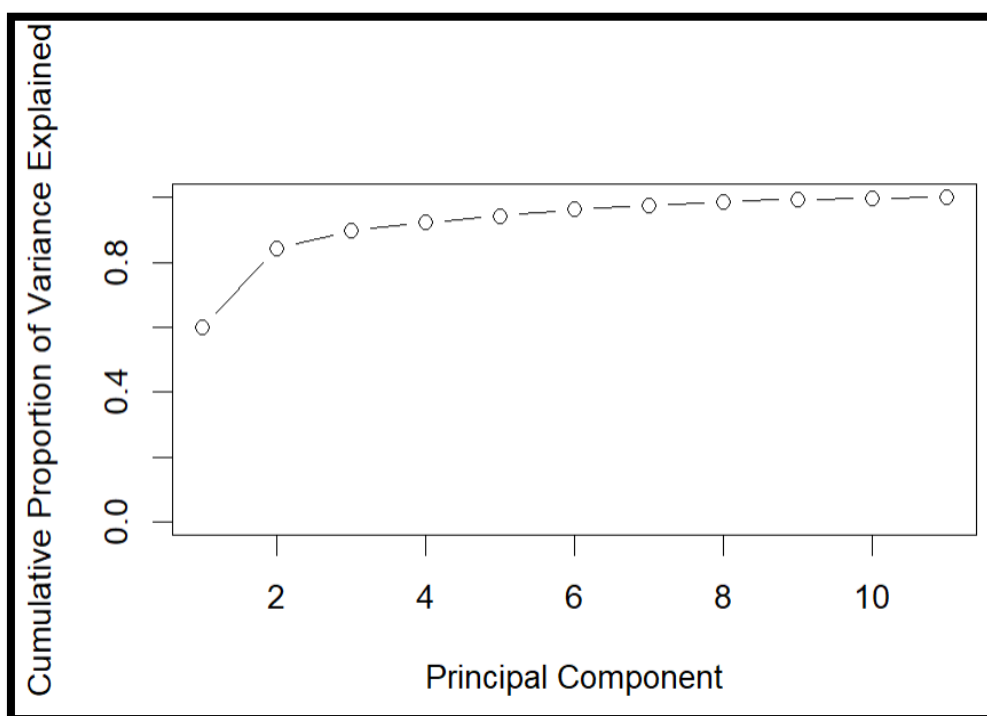
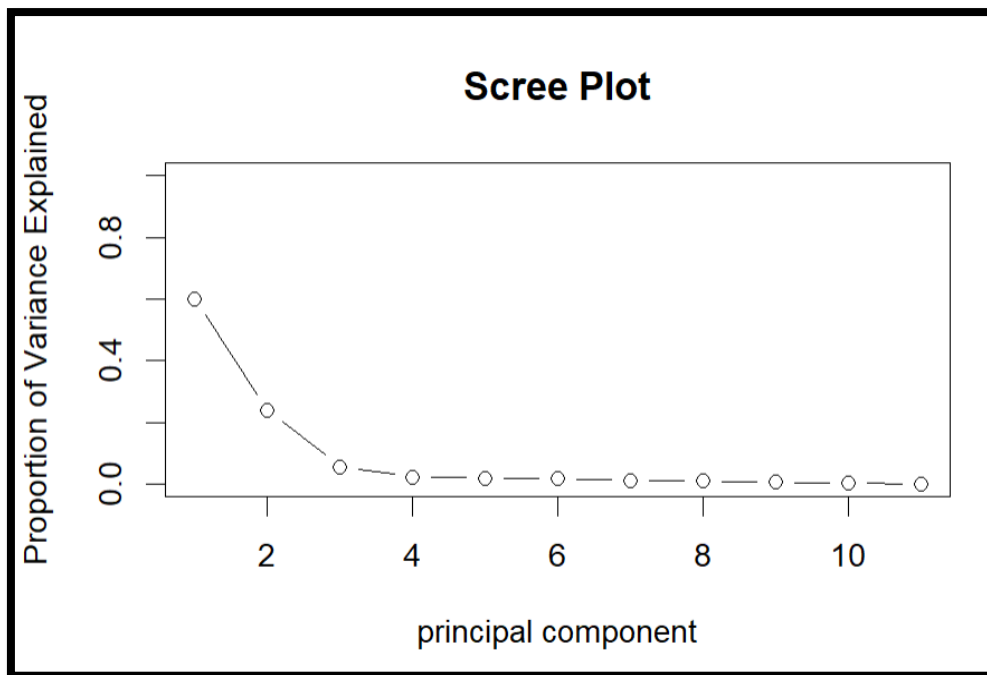
rpart.model = rpart(displacement ~ .,
                    data = train.data, method = "anova")

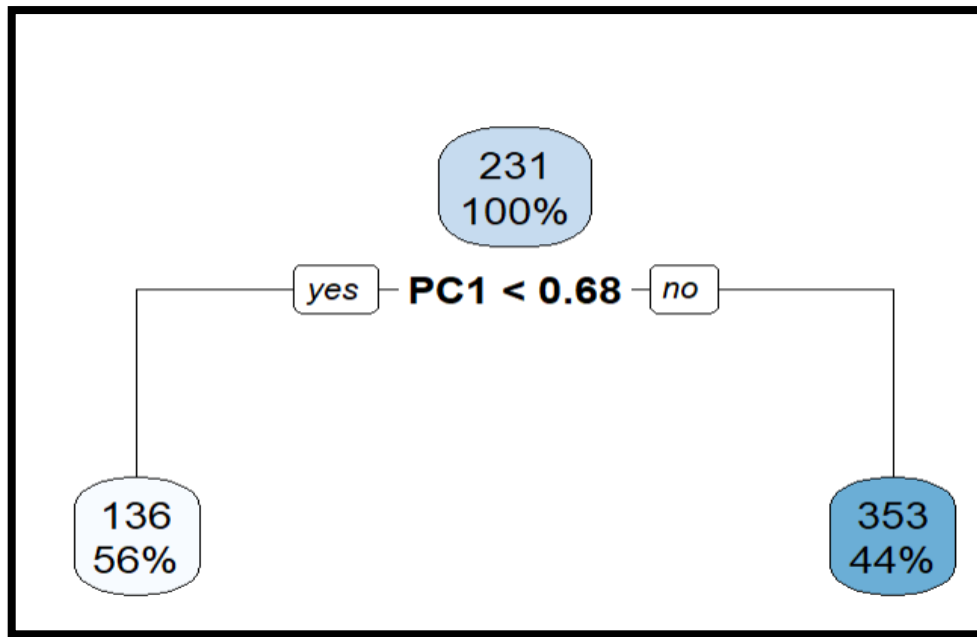
rpart.plot(rpart.model)

```

### Output:







**9c: Visualize the data in the reduced-dimensional space.****#Generate data**

```
m=50
```

```
n=100
```

```
frac.gaps <- 0.5 # the fraction of data with NaNs
```

```
N.S.ratio <- 0.25 # the Noise to Signal ratio for adding noise to data
```

```
x <- (seq(m)*2*pi)/m
```

```
t <- (seq(n)*2*pi)/n
```

**#True field**

```
Xt <-
```

```
  outer(sin(x), sin(t)) +  
  outer(sin(2.1*x), sin(2.1*t)) +  
  outer(sin(3.1*x), sin(3.1*t)) +  
  outer(tanh(x), cos(t)) +  
  outer(tanh(2*x), cos(2.1*t)) +  
  outer(tanh(4*x), cos(0.1*t)) +  
  outer(tanh(2.4*x), cos(1.1*t)) +  
  tanh(outer(x, t, FUN="+")) +  
  tanh(outer(x, 2*t, FUN="+"))
```

```
Xt <- t(Xt)
```

**#PCA**

```
res <- prcomp(Xt, center = TRUE, scale = FALSE)
```

```
names(res)
```

```
res$sdev
```

```
length(res$sdev)
```

```
res$rotation
```

```
dim(res$rotation)
```

```
res$x
```

```
dim(res$x)
```

```
pc.use <- 3 # explains 93% of variance
```

```
trunc <- res$x[,1:pc.use] %*% t(res$rotation[,1:pc.use])
```

**#and add the center (and re-scale) back to data**

```
if(all(res$scale != FALSE)){
```

```
  trunc <- scale(trunc, center = FALSE, scale=1/res$scale)
```

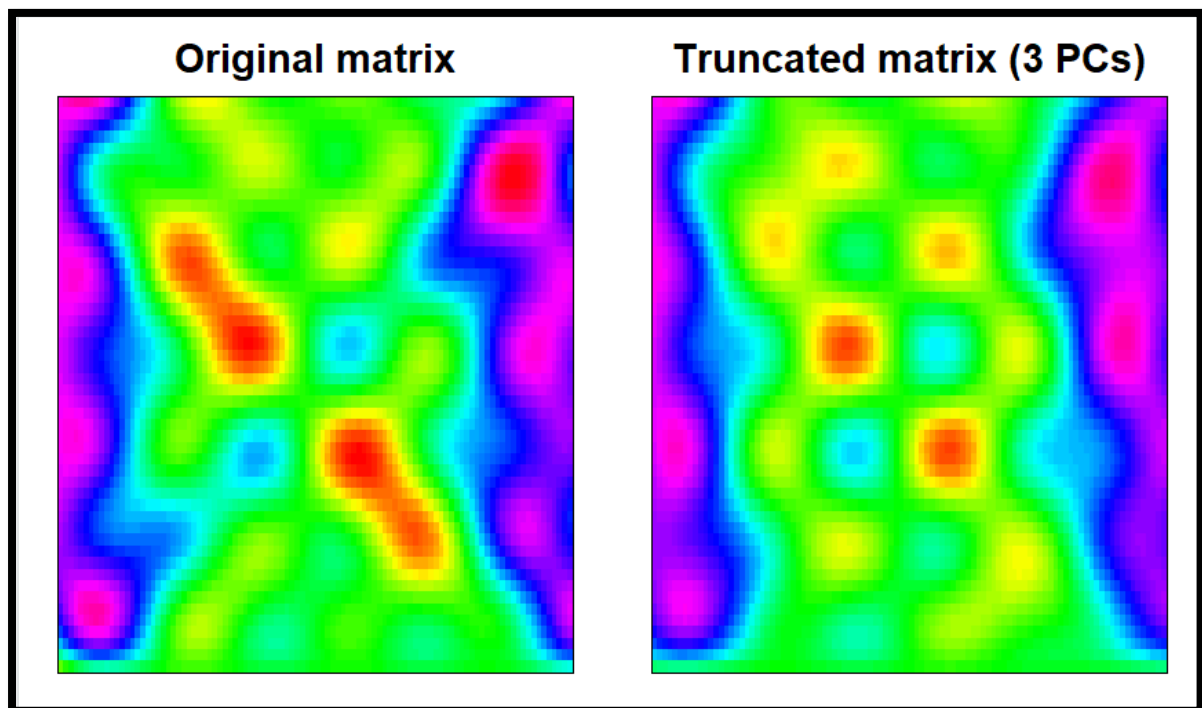
```
}
```

```
if(all(res$center != FALSE)){
```

```
  trunc <- scale(trunc, center = -1 * res$center, scale=FALSE)
```

```
}  
dim(trunc); dim(Xt)  
RAN <- range(cbind(Xt, trunc))  
BREAKS <- seq(RAN[1], RAN[2],,100)  
COLS <- rainbow(length(BREAKS)-1)  
par(mfcol=c(1,2), mar=c(1,1,2,1))  
image(Xt, main="Original matrix", xlab="", ylab="", xaxt="n", yaxt="n",  
breaks=BREAKS, col=COLS)  
box()  
image(trunc, main="Truncated matrix (3 PCs)", xlab="", ylab="", xaxt="n",  
yaxt="n", breaks=BREAKS, col=COLS)  
box()
```

**Output:**





## Practical No 10

**Aim: 10A Create a meaningful Visualization using data visualization tools.**

**Aim: 10B Combine multiple visualizations to tell a compelling data story.**

**Code:**

```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import pandas as pd
from sklearn.cluster import KMeans

df = pd.DataFrame({
    'gender': ['Male', 'Female', 'Female', 'Male', 'Male'],
    'churn': ['Yes', 'No', 'Yes', 'No', 'Yes'],
    'age': [25, 42, 32, 30, 35],
    'service_type': ['Permanent Staffing', 'Contract Staffing', 'Part-Time Staffing',
                    'Freelance', 'Temporary'],
    'tenure': [2, 6, 10, 5, 7]
})

# 1. Churn rate by gender
sns.countplot(data=df, x='gender', hue='churn')
plt.title("Churn rate by gender")
plt.show()

# 2. Churn rate by age group (Plotly)
fig = px.histogram(
    df,
    x='age',
    color='churn',
    nbins=20,
    histnorm='percent'
)
fig.update_layout(
    title='Churn rate by age group',
    xaxis_title='Age',
    yaxis_title='Percentage of customers'
)
fig.show()

# 3. Correlation matrix
# Convert churn to numeric if needed
df['churn_numeric'] = df['churn'].map({'Yes': 1, 'No': 0})
correlation_matrix = df.select_dtypes(include='number').corr()

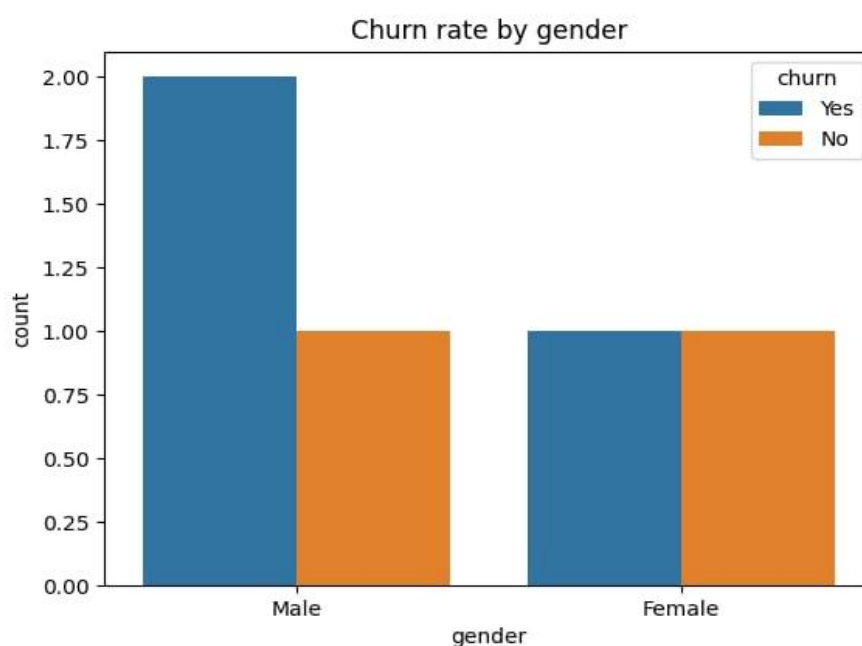
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation matrix')
plt.show()

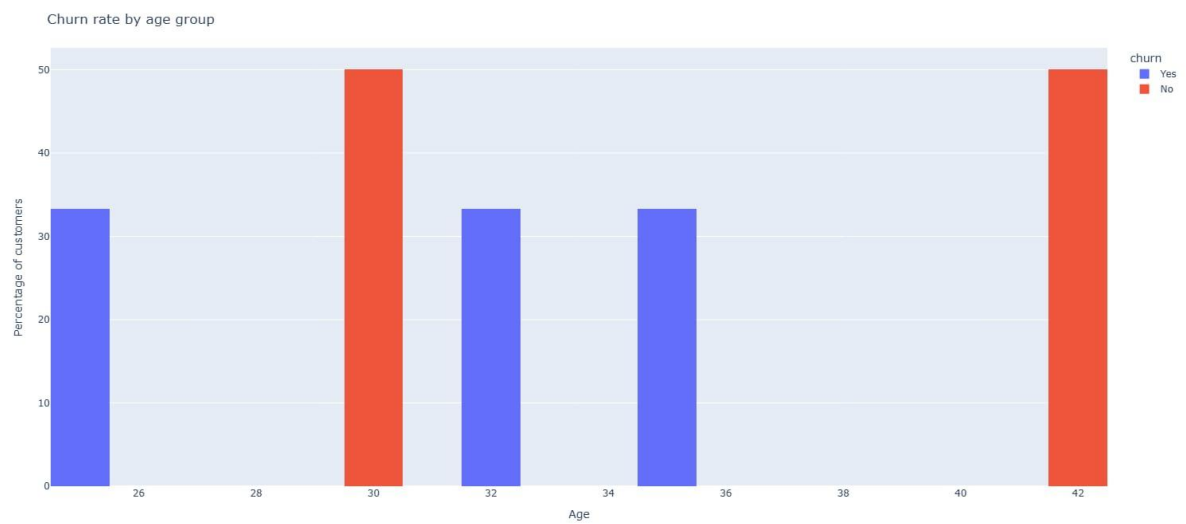
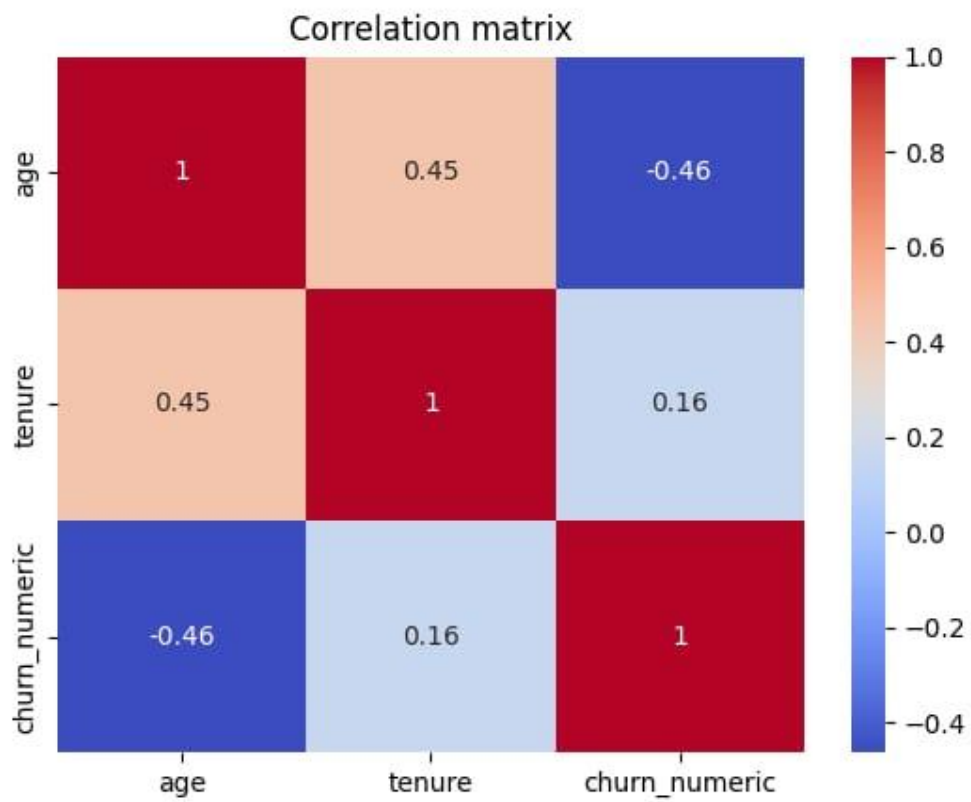
# 4. Customer tenure vs churn (Plotly)
```

```
fig = px.scatter(
    df,
    x='tenure',
    y='churn_numeric',
    color='churn'
)
fig.update_layout(
    title='Customer Tenure vs Churn',
    xaxis_title='Tenure',
    yaxis_title='Churn (1 = Yes, 0 = No)'
)
fig.show()
# 5. Select numerical features
X = df[['age', 'tenure']]

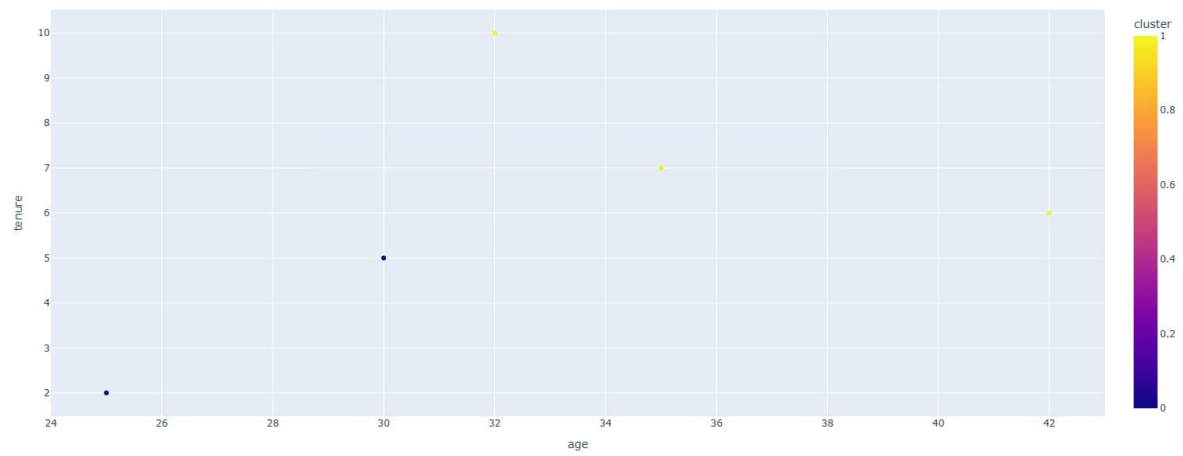
# Apply KMeans
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X)
fig = px.scatter(
    df,
    x='age',
    y='tenure',
    color='cluster',
    title='Customer Segmentation using KMeans'
)
print(df.groupby('cluster')[['age', 'tenure']].mean())
fig.show()
```

OUTPUT:





Customer Segmentation using KMeans



Customer Tenure vs Churn

